NTNU
Kunnskap for en bedre verden

Department of Mathematical Sciences

Learning Neural Junction Models for Macroscopic Traffic Networks

Author:
Casper Lindeman

Supervisors:
Kjetil Olsen Lye
Knut-Andreas Lie
Franz Georg Fuchs

December, 2025

Contents

List of Figures

List of Tables

1 Introduction

Traffic flow on road networks arises from interactions between vehicle motion on individual roads and the redistribution of traffic at intersections. Even at a *macroscopic* level, where traffic is described by aggregated quantities such as density and flow, the resulting dynamics can be complex. Mathematical models are therefore widely used to analyze and simulate traffic behavior for both theoretical and practical purposes.

A common class of traffic models describes the evolution of traffic density through *conservation laws*, which encode vehicle motion along a road while preserving the total number of cars. These models are computationally efficient and capture key phenomena such as congestion formation and wave propagation. On a single road, the governing equations fully determine the traffic dynamics. In a road network, however, additional modeling choices are required at intersections, where traffic from multiple roads interacts.

Intersection behavior is typically described by explicit analytical junction models, which prescribe how traffic is transferred between incoming and outgoing roads [1, 2]. These models play a central role in determining the global network dynamics, as they control how congestion and disturbances propagate across the network. At the same time, junction behavior depends on factors such as driver decisions, local priorities, infrastructure layout, and traffic control strategies, which are difficult to model explicitly. As a result, analytical junction models rely on simplifying assumptions and may only approximate observed traffic behavior under specific conditions.

Data-driven and machine-learning-based methods have recently been applied to the modeling of complex dynamical systems. In particular, *physics-informed neural networks* (PINNs) and related approaches seek to combine physical models with observational data, either by enforcing governing equations during training or by learning unknown model components [3]. These methods provide a mechanism for incorporating data while retaining elements of the underlying physical structure. PINN-based approaches have also been proposed for macroscopic traffic flow models, for example for traffic state and queue profile estimation using differentiable programming frameworks [4].

At the same time, many components of classical traffic models are already well understood and accurately approximated by established numerical solvers. The dynamics along individual road segments can be reliably described by conservation laws with well-studied *stability* and convergence properties. Replacing these components by fully data-driven models is not necessary. Instead, learning can be introduced only at those parts of the model that are not fixed by the conservation law.

This report adopts a *hybrid* modeling approach that combines physics-based and data-driven components in a targeted manner. The traffic dynamics on each road are described by a fixed conservation law model, while the junction behavior is represented by a neural network. In this way, the overall structure of the traffic model is preserved, and learning is confined to the component that is most sensitive to modeling assumptions. Related hybrid physics–data approaches have been studied for *inverse problems* in *hyperbolic* partial differential equations (PDEs), where neural networks are combined with classical numerical solvers to learn unknown model components, see for example [5].

The main contribution of this report is the formulation and numerical investigation of such a hybrid road–network model. We study the behavior of neural junction models embedded in a numerical traffic solver and analyze its performance under different observation and training scenarios, ranging from idealized settings to sparse measurements. The results aim to clarify both the potential and the limitations of learning junction behavior from data.

2 Traffic flow model

Traffic flow can be modeled at different levels of detail. Two main types of models are used, *microscopic* and macroscopic models. Microscopic models describe each individual vehicle and

its interaction with nearby cars. The state of the system is then given by the positions and velocities of all vehicles, and their motion is governed by rules for acceleration and braking, as well as interaction mechanisms such as lane changing, turning, merging, and yielding. Such rules are typically formulated through car-following and lane-changing models that describe how drivers respond to local traffic conditions. Microscopic models are well suited for detailed studies of small traffic systems where individual driving behaviour is important. For general introductions and comprehensive overviews of microscopic traffic modelling, we refer to [6, 7].

Macroscopic models do not follow individual vehicles. Instead, traffic is described by averaged quantities such as the vehicle density and the mean velocity, which are treated as functions of space and time. These models are inspired by fluid dynamics and are computationally much less expensive than microscopic models. They are therefore particularly useful for simulations on long road segments and in larger road networks. Since they are based on averaged quantities, they do not resolve individual driving behaviour and may develop discontinuities in the form of shock waves.

The two modeling approaches are closely related. Macroscopic models can be interpreted as arising from averages of the behaviour of many vehicles described by microscopic models, while microscopic models can be seen as detailed representations of the macroscopic dynamics. The choice between the two depends on the goal of the simulation. In this work, we use a macroscopic model and apply it to relatively small and simple road networks in order to study its mathematical structure, the behaviour at junctions, and the associated numerical methods, even though the same framework is mainly used for large-scale simulations in practice.

In this report, the macroscopic dynamics are described by a conservation law formulated as a PDE. The key idea is that the number of vehicles in any road segment can only change because vehicles enter or leave the segment. This transport mechanism is encoded through a *flux* function, which specifies how many vehicles pass a point per unit time. In the next subsection, we derive the specific model used throughout this report from basic conservation principles. We then discuss mathematical properties that are essential both for understanding traffic flow and for constructing reliable numerical schemes, and show how the model can be extended from a single road to a road network.

2.1 The Lighthill–Whitham–Richards traffic model

We consider traffic on a single one-dimensional road. Since we use a macroscopic model, we do not follow individual cars. Instead, we approximate the traffic as a compressible continuum. In this description, the amount of traffic at position $\bar{x}$ and time t is represented by the density $\bar{\rho}(\bar{x}, t)$ of cars. In the original theoretical formulations, this density is treated as a pointwise quantity. In practice, however, $\bar{\rho}(\bar{x}, t)$ is interpreted as a local average number of cars per unit length over a short road segment around $\bar{x}$, which allows the model to be compared with measured traffic data. Let $\bar{\rho}$, $\bar{v}$ and $\bar{x}$ denote the physical density, velocity and position. These quantities satisfy

$$0 \le \bar{\rho} \le \rho_{\max}, \qquad 0 \le \bar{v} \le v_{\max}, \qquad 0 \le \bar{x} \le L,$$

where $\rho_{\max}$ denotes the bumper-to-bumper density and depends on the number of lanes on the road, taking the value $\rho_{\max} = 1$ for a single-lane road and $\rho_{\max} = 2$ for a two-lane road. The parameter $v_{\max}$ is the speed limit, and L is the total physical length of the road.

The mass in a segment $[\bar{x}, \bar{x} + \Delta\bar{x}]$ at time t is $\int_{\bar{x}}^{\bar{x}+\Delta\bar{x}} \bar{\rho}(y, t)\, dy$. We assume that mass is neither created nor destroyed inside the segment. The only way to change the mass is through flow across the boundaries. If $\bar{f}(\bar{x}, t)$ denotes the mass-flux at position $\bar{x}$ and time t, then conservation of mass gives

$$\frac{d}{dt} \int_{\bar{x}}^{\bar{x}+\Delta\bar{x}} \bar{\rho}(y, t)\, dy = \bar{f}(\bar{x}, t) - \bar{f}(\bar{x} + \Delta\bar{x}, t). \tag{1}$$

In other words, the temporal change of mass in the segment equals the inflow flux minus the outflow flux.

To describe how the cars moves, we need an expression for the flux. If a fluid with density $\bar{\rho}$ is transported with velocity $\bar{v}$, then the amount crossing a point per unit time is

$$\bar{f}(\bar{x}, t) = \bar{\rho}(\bar{x}, t)\bar{v}(\bar{x}, t).$$

This is the standard definition of flux for a transported quantity.

Since we want to solve for $\bar{\rho}$ as the conserved quantity, there are two main approaches. One option is to keep a single equation by specifying the velocity in advance, either as a function of the density $\bar{v}(\bar{\rho})$ or as a prescribed function $\bar{v}(\bar{x}, t)$. In both cases, the only unknown in the system is $\bar{\rho}(\bar{x}, t)$. The other option is to let the velocity be an additional unknown and add a separate equation for it, so that we solve a system of equations instead of just one. This is often done by introducing a second conserved quantity. We will take a quick look at how this can be done in Section 2.7.

We assume the velocity is entirely dependent on the density. This assumption is reasonable in a traffic setting, since the speed of the cars is largely determined by how crowded the road is. Vehicles move freely at low densities and are forced to slow down as the density increases. The result is the flux relation

$$\bar{f}(\bar{\rho}) = \bar{\rho}\bar{v}(\bar{\rho}). \tag{2}$$

With the flux expressed in terms of the density, we can now return to the integral mass conservation relation (1). We assume that $\bar{\rho} \in C^1([0, L] \times [0, T])$ and $\bar{v} \in C^1([0, \rho_{\max}])$ are sufficiently smooth. Using the flux presented in (2), we get $\bar{f} \in C^1([0, L] \times [0, T])$, as it is a composition of two continuously differentiable functions. In other words, $\bar{\rho}$, $\bar{v}$, and $\bar{f}$ are all continuously differentiable for all possible points on the road at all possible times. This ensures all the derivatives used below are well defined. By the Leibniz rule, the left-hand side of (1) becomes

$$\frac{d}{dt}\int_{\bar{x}}^{\bar{x}+\Delta\bar{x}} \bar{\rho}(y, t)\, dy = \int_{\bar{x}}^{\bar{x}+\Delta\bar{x}} \frac{\partial}{\partial t}\bar{\rho}(y, t)\, dy.$$

For the right-hand side, we use the fundamental theorem of calculus. Since $\bar{f}(\bar{\rho})$ depends on $\bar{x}$ through $\bar{\rho}(\bar{x}, t)$, we have

$$\bar{f}(\bar{x}, t) - \bar{f}(\bar{x} + \Delta\bar{x}, t) = -\int_{\bar{x}}^{\bar{x}+\Delta\bar{x}} \frac{\partial}{\partial y}\bar{f}(\bar{\rho}(y, t))\, dy.$$

Substituting these expressions into (1) gives

$$\int_{\bar{x}}^{\bar{x}+\Delta\bar{x}} \frac{\partial}{\partial t}\bar{\rho}(y, t)\, dy = -\int_{\bar{x}}^{\bar{x}+\Delta\bar{x}} \frac{\partial}{\partial y}\bar{f}(\bar{\rho}(y, t))\, dy,$$

and inserting (2) gives the equivalent relation

$$\int_{\bar{x}}^{\bar{x}+\Delta\bar{x}} \left[\bar{\rho}_t(y, t) + (\bar{\rho}(y, t)\bar{v}(\bar{\rho}(y, t)))_x\right] dy = 0.$$

Since this identity holds for every interval $[\bar{x}, \bar{x} + \Delta\bar{x}]$, the expression inside the integrand must be zero everywhere. We therefore obtain the differential form of the conservation law,

$$\bar{\rho}_t + (\bar{\rho}\bar{v}(\bar{\rho}))_x = 0. \tag{3}$$

This is the *Lighthill–Whitham–Richards* (LWR) model [8, 9]. In the traffic-flow literature, it is classified as a *first-order* model, meaning that the density is the only unknown variable, while the velocity is prescribed directly as a function of the density and not determined by a separate equation.

The LWR model is a special case of a general class of PDEs known as hyperbolic conservation laws, which describe the evolution of conserved quantities transported through space by a flux. In general, such a conservation law can be written in the form

$$\begin{cases} u_t + \nabla_x f(u) = 0, & (x, t) \in \Omega \times \mathbb{R}_+, \\ u(x, 0) = u_0(x), & x \in \Omega, \end{cases} \tag{4}$$

where $\Omega \subset \mathbb{R}^d$ is the spatial domain, d denotes the spatial dimension, and $u : \Omega \times \mathbb{R}_+ \to \mathbb{R}^N$ represents one or several conserved quantities. The function u_0 is the initial condition and $f : \mathbb{R}^N \to \mathbb{R}^{N\times d}$ is the flux function. If $N = 1$, the equation is called a *scalar* conservation law. The conservation law is said to be hyperbolic if the associated linearized system admits real eigenvalues for every admissible state $u \in \mathbb{R}^N$. In the scalar case $N = 1$, this condition is always satisfied. The system is supplemented with suitable boundary conditions on $\partial\Omega$.

Hyperbolic conservation laws arise in many physical applications and describe the transport of conserved quantities such as mass, momentum, and energy. The key feature is that these quantities can only change through fluxes across the boundary of a region. In gas dynamics, the one-dimensional Euler equations describe the conservation of mass, momentum, and energy in a compressible gas. In shallow-water theory, systems of conservation laws model the motion of water in rivers and coastal regions through the conservation of water height and momentum. In transport problems, advection equations describe how quantities such as temperature or pollutants are carried by a given flow field. The LWR equation belongs to this same class of models, with the vehicle density as the conserved quantity and the traffic flux given by the product of density and velocity. For general introductions to hyperbolic conservation laws and their applications, we refer to standard textbooks such as [10].

For the numerical simulations, we work with dimensionless variables $\rho, v, x \in [0, 1]$. They are related to the physical quantities through the *scaling*

$$\bar{\rho} = \rho_{\max}\rho, \qquad \bar{v} = v_{\max}v, \qquad \bar{x} = L\,x. \tag{5}$$

The corresponding dimensionless flux is defined by scaling the physical flux as

$$\bar{f}(\bar{\rho}) = \rho_{\max}v_{\max}f(\rho).$$

Time is left unscaled. Inserting the scaled variables into (3) we get the dimensionless LWR model

$$\rho_t + \gamma(\rho v(\rho))_x = 0, \tag{6}$$

where $\gamma = v_{\max}/L$. From this point on, we only work with the scaled variables and drop the bars.

The model is not complete yet, since we still need a relation between velocity and density. Many choices are possible. A general observation is that when the traffic becomes congested, vehicles move slowly, while in light traffic, they can move at or near the speed limit. The relation between velocity and density can therefore be modeled in several ways, depending on how we want to capture driver behavior. A simple and commonly used choice is the *Greenshields model*, which assumes a linear relation between velocity and density,

$$v(\rho) = 1 - \rho. \tag{7}$$

The corresponding flux function then becomes

$$f(\rho) = \rho(1 - \rho), \tag{8}$$

This means that the total flow of cars is highest when the road is half full. In Figure 1, we plot the velocity and flux functions for the Greenshields model. Inserting (7) into (6) gives the complete model on the domain $x \in (0, 1)$ in the form of the initial–boundary value problem

$$\begin{cases} \rho_t + \gamma(1 - 2\rho)\rho_x = 0, & x \in (0, 1),\ t > 0, \\ \rho(x, 0) = \rho_0(x), & x \in [0, 1], \end{cases} \tag{9}$$

supplemented with boundary conditions at $x = 0$ and $x = 1$ prescribed through the boundary fluxes. This is the model used throughout this report.

2.2 Characteristics

Having derived the LWR equation as a scalar hyperbolic conservation law, we now turn to the analysis of its smooth solutions. For equations of this type, the standard analytical framework is

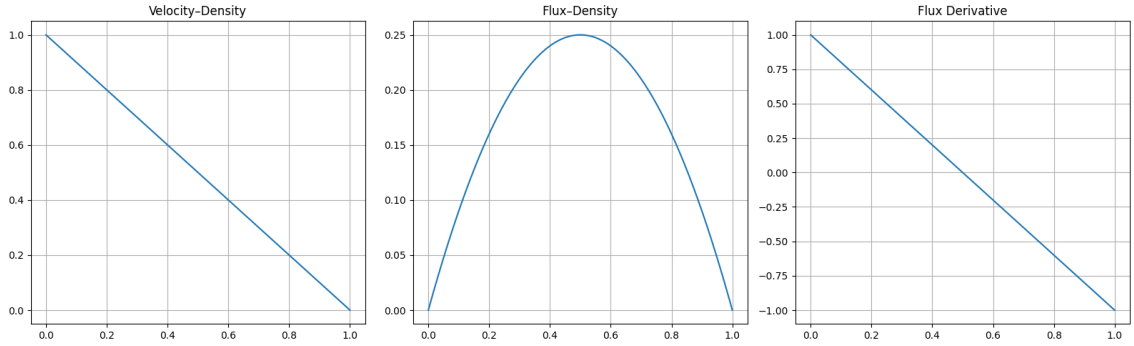

Figure 1: The velocity, flux and flux derivative as functions of density for the Greenshields model.

provided by the *method of characteristics*, which describes how the solution is transported along curves in the (x,t)–plane determined by the flux.

To see this, we consider a curve in the (x,t)–plane given by $(x(t),t)$, starting from an initial point $(x_0,0)$ where the density is $\rho_0(x_0)$, with $\rho_0 \in C^1([0,1])$. Along this curve the density evolves according to the chain rule,

$$\frac{d\rho(x(t),t)}{dt} = \rho_t + \frac{dx(t)}{dt}\,\rho_x. \tag{10}$$

The right-hand side has the same structure as the LWR model (9). If we choose the curve $(x(t),t)$ so that (10) matches the PDE term by term, then the total derivative vanishes. This happens precisely when

$$\frac{dx(t)}{dt} = \gamma f'(\rho) = \gamma(1-2\rho). \tag{11}$$

With this choice, (10) becomes

$$\frac{d\rho(x(t),t)}{dt} = 0.$$

Thus, the density remains constant along the curve, meaning $\rho(x(t),t) = \rho_0(x_0)$. Since the characteristic speed in (11) depends only on ρ, it is also constant along the curve. Solving the ordinary differential equation (ODE) in (11) gives the parametric form

$$x(t) = x_0 + \gamma(1-2\rho_0(x_0))\,t. \tag{12}$$

These curves are the characteristics of the scaled LWR model using the Greenshields velocity. Each point in the initial data generates its own characteristic and the density value is carried along that line. In this model, the characteristics are straight lines, and as long as they do not overlap, the solution stays smooth and is completely determined by these lines. This behaviour is typical for scalar hyperbolic conservation laws. As long as the solution is smooth, the conserved quantity is transported unchanged along the characteristic curves.

A discontinuity forms when two characteristic lines meet. In that situation, different initial points attempt to assign different density values to the same location (x,t), and the classical pointwise interpretation of the PDE breaks down. The solution therefore develops a jump in the density, called a *shock*. In traffic flow, such a shock represents the sudden transition from fast-moving to slow-moving vehicles, which is the formation of a traffic jam.

To see when two characteristics intersect, note that each characteristic travels with the constant speed given in (11). Two characteristics starting at points $x_1 < x_2$ will meet if the one issued from x_1 moves faster than the one from x_2. This happens if and only if

$$f'(\rho_0(x_1)) > f'(\rho_0(x_2)).$$

If this inequality holds, the intersection occurs at the finite time

$$t^* = \frac{x_2 - x_1}{\gamma\big(f'(\rho_0(x_1)) - f'(\rho_0(x_2))\big)}.$$

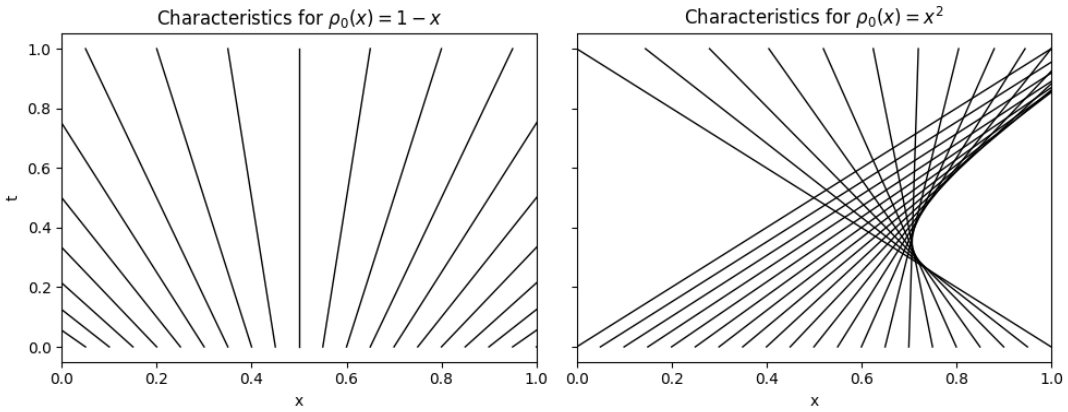

Figure 2: Characteristics of (9) for two smooth initial conditions. Left: $\rho_0(x) = 1-x$, giving diverging characteristics and a smooth solution. Right: $\rho_0(x) = x^2$, giving intersecting characteristics and shock formation.

For smooth initial data, the characteristic speed varies smoothly with x_0. A shock therefore forms in regions where this speed decreases as we move to the right, which is equivalent to the condition

$$\frac{d}{dx_0}\big(\gamma f'(\rho_0(x_0))\big) < 0.$$

Differentiating gives

$$f''(\rho_0(x_0))\,\rho_0'(x_0) < 0.$$

Since the flux is concave, see (49), this inequality holds whenever $\rho_0'(x_0) > 0$. Any region where the initial density is increasing therefore produces a shock in finite time. In Figure 2 we have plotted the characteristics of (9) for two different smooth initial conditions on $x \in [0, 1]$. For the left panel, we use the decreasing initial data $\rho_0(x) = 1-x$, for which $\rho_0'(x) < 0$ and the characteristics spread out, so no shock is formed. For the right panel, we use the increasing initial data $\rho_0(x) = x^2$, for which $\rho_0'(x) > 0$ and the characteristics intersect, leading to the formation of a shock, since different characteristics try to impose different density values at the same point.

Thus, even with smooth initial data, the solution of the LWR model may develop a discontinuity created when characteristics intersect. At such points, the spatial derivative ρ_x is not defined and the PDE cannot be interpreted in the classical pointwise sense. Moreover, solutions may start from discontinuous initial data, in which case jumps are present from the beginning. In both situations, the classical formulation of the LWR equation is insufficient. To describe solutions beyond the formation of a shock, and to allow for discontinuous initial profiles, we need a more general notion of solution, one that admits jumps but still respects the underlying conservation law.

2.3 Weak solutions and Rankine Hugoniot

In the previous section, we showed that solutions of the LWR model may develop discontinuities in finite time even for smooth initial data. At such points, the spatial derivative ρ_x is not defined and the differential form of the equation cannot be interpreted pointwise. However, the underlying principle of conservation of cars still remains valid. To allow for solutions with discontinuities we therefore work with the integral form of the equation and introduce the notion of *weak solutions*.

Let $\varphi \in C_c^1(\mathbb{R} \times \mathbb{R}_+)$ be a smooth test function with compact support, meaning that $\varphi(x, t) = 0$ outside a bounded region of the (x, t)–plane. Multiplying the conservation law (9) by φ and integrating over $\mathbb{R} \times (0, \infty)$ gives

$$\iint_{\mathbb{R}\times(0,\infty)} (\rho_t + \gamma f(\rho)_x)\,\varphi\,dx\,dt = 0.$$

This integral identity represents conservation of cars over an arbitrary space–time control volume selected by the test function φ. It states that any change in the number of cars inside such a region

is exactly balanced by the flux through its boundary. Integrating by parts in both time and space, and using that φ has compact support, so that boundary terms vanish except at $t = 0$, we obtain

$$\iint_{\mathbb{R}\times(0,\infty)} \rho\,\varphi_t + \gamma f(\rho)\,\varphi_x \,dx\,dt + \int_{\mathbb{R}} \rho_0(x)\,\varphi(x,0)\,dx = 0. \tag{13}$$

A function $\rho(x,t)$ that satisfies this identity for all such test functions φ is called a weak solution. No derivatives of ρ appear in this formulation, so ρ is not required to be differentiable and may contain discontinuities. If ρ is smooth, the weak formulation is equivalent to the classical differential equation. Weak solutions therefore extend the notion of solution beyond the formation of shocks.

In the characteristic description from Section 2.2, a shock appears when characteristic curves intersect. After this intersection, the solution is no longer transported along a family of smooth characteristic lines, but instead develops a moving interface in the (x,t)–plane separating two smooth states. This interface traces out a curve $(s(t),t)$ in the (x,t)–plane, which represents the trajectory of the shock and thus the motion of a traffic jam through the flow. To determine how such a discontinuity propagates in time, we now derive a necessary condition that any weak solution with a single discontinuity must satisfy. This condition is known as the *Rankine–Hugoniot condition*.

Consider a weak solution $\rho(x,t)$ consisting of two smooth states separated by a moving discontinuity. That is,

$$\rho(x,t) = \begin{cases} \rho_L(x,t), & x < s(t), \\ \rho_R(x,t), & x > s(t), \end{cases} \tag{14}$$

where $x = s(t)$ describes the position of the discontinuity. The functions $\rho_L, \rho_R \in C^1$ satisfy the conservation law $\rho_t + \gamma f(\rho)_x = 0$ in the classical sense on the regions $x < s(t)$ and $x > s(t)$, respectively.

Let $\varphi \in C_c^1(\mathbb{R}\times(0,\infty))$ be chosen such that its support intersects the curve $x = s(t)$ but does not reach the line $t = 0$. Inserting φ and (14) into (13), we obtain

$$\begin{aligned} 0 &= \iint_{\mathbb{R}\times(0,\infty)} \big(\rho\,\varphi_t + \gamma f(\rho)\,\varphi_x\big)\,dx\,dt + \int_{\mathbb{R}} \rho_0(x)\,\varphi(x,0)\,dx \\ &= \iint_{x<s(t)} \big(\rho_L\,\varphi_t + \gamma f(\rho_L)\,\varphi_x\big)\,dx\,dt + \iint_{x>s(t)} \big(\rho_R\,\varphi_t + \gamma f(\rho_R)\,\varphi_x\big)\,dx\,dt. \end{aligned} \tag{15}$$

Since the support of φ does not reach the line $t = 0$, we have $\varphi(x,0) = 0$ and thus the initial condition term vanishes. The remaining space–time integral is then split using the piecewise definition (14) of ρ into the two regions separated by the moving discontinuity.

Since ρ_L is a classical solution on $x < s(t)$, it is differentiable and we may integrate by parts in both t and x. Since φ has compact support in time, there exist $0 < t_1 < t_2$ such that $\operatorname{supp}\varphi \subset \mathbb{R}\times(t_1,t_2)$. We therefore obtain

$$\begin{aligned} \iint_{x<s(t)} \big(\rho_L\varphi_t + \gamma f(\rho_L)\varphi_x\big)\,dx\,dt &= \Big[\int_{-\infty}^{s(t)} \rho_L(x,t)\varphi(x,t)\,dx\Big]_{t_1}^{t_2} - \iint_{x<s(t)} (\rho_L)_t\varphi\,dx\,dt \\ &\quad + \int_{t_1}^{t_2} \Big[\gamma f(\rho_L(x,t))\varphi(x,t)\Big]_{-\infty}^{s(t)} dt - \iint_{x<s(t)} \gamma(f(\rho_L))_x\varphi\,dx\,dt \\ &\overset{\text{(i)}}{=} \int_{t_1}^{t_2} \big(\gamma f(\rho_L(s(t),t)) - s'(t)\rho_L(s(t),t)\big)\varphi(s(t),t)\,dt \\ &\quad - \iint_{x<s(t)} \big(\underbrace{(\rho_L)_t + \gamma(f(\rho_L))_x}_{=0}\big)\varphi\,dx\,dt \\ &= \int_{t_1}^{t_2} \big(\gamma f(\rho_L(t)) - s'(t)\rho_L(t)\big)\varphi(s(t),t)\,dt. \end{aligned} \tag{16}$$

In step (i) we merged the two space-time integrals and used that φ has compact support, so the boundary terms at $t = t_1, t_2$ and $x \to -\infty$ vanish. Moreover, the remaining space-time integral vanishes since ρ_L satisfies the conservation law $\rho_t + \gamma f(\rho)_x = 0$ on $x < s(t)$.

Similarly, on the region $x > s(t)$, where ρ_R is a classical solution, we obtain

$$\iint_{x>s(t)} \big(\rho_R\varphi_t + \gamma f(\rho_R)\varphi_x\big)\,dx\,dt = \int_{t_1}^{t_2} \big(-\gamma f(\rho_R(t)) + s'(t)\rho_R(t)\big)\varphi(s(t),t)\,dt. \tag{17}$$

Inserting (16) and (17) into (15) gives

$$\int_{t_1}^{t_2} \Big(\gamma\big(f(\rho_R(t)) - f(\rho_L(t))\big) - s'(t)\big(\rho_R(t) - \rho_L(t)\big)\Big)\,\varphi(s(t),t)\,dt = 0.$$

Since $\varphi(s(t),t)$ can be chosen arbitrarily, the expression in parentheses must vanish identically for all t. Hence,

$$\gamma\big(f(\rho_R(t)) - f(\rho_L(t))\big) - s'(t)\big(\rho_R(t) - \rho_L(t)\big) = 0.$$

Equivalently,

$$\frac{ds(t)}{dt} = \gamma\,\frac{f(\rho_R) - f(\rho_L)}{\rho_R - \rho_L}. \tag{18}$$

This relation is the Rankine–Hugoniot condition. It describes how the path of the shock $(s(t),t)$ evolves in the (x,t)–plane for any weak solution of the form (14) and shows that the shock speed depends only on the left and right states ρ_L and ρ_R.

2.4 Entropy condition

The weak formulation (13) and the Rankine–Hugoniot condition (18) describe how discontinuities may appear and propagate in solutions of (9). However, they do not guarantee uniqueness. In this section we show this explicitly by constructing two different weak solutions of the same initial value problem. This motivates the introduction of an *entropy condition*, which is needed to select the physically relevant solution.

We consider the conservation law (9) posed on the whole real line $x \in \mathbb{R}$, so that boundary effects can be ignored. We prescribe piecewise constant initial data with a single jump at the origin,

$$\rho(x,0) = \begin{cases} \rho_L, & x < 0, \\ \rho_R, & x > 0, \end{cases} \tag{19}$$

where ρ_L and ρ_R are constants. This initial value problem is called the *Riemann problem*. It plays a fundamental role in the theory of conservation laws and is a basic building block for the numerical methods presented later in Section 3.

One weak solution of the Riemann problem (19) is given by a single moving discontinuity separating two constant states,

$$\rho(x,t) = \begin{cases} \rho_L, & x < s(t), \\ \rho_R, & x > s(t), \end{cases} \tag{20}$$

where the curve $(s(t),t)$ denotes the location of the jump. The speed of the shock is determined by the Rankine–Hugoniot condition derived in Section 2.3. For the choice $\rho_L = 0.95$ and $\rho_R = 0.15$, the corresponding characteristic structure is shown in the left panel of Figure 3, where the thick line represents the curve $(s(t),t)$.

Another weak solution of the same Riemann problem is obtained by resolving the initial jump through a continuous transition. This solution has the self–similar form

$$\rho(x,t) = R\Big(\frac{x}{t}\Big), \qquad R(\xi) = \begin{cases} \rho_L, & \xi \le \gamma f'(\rho_L), \\ (f')^{-1}(\xi/\gamma), & \gamma f'(\rho_L) < \xi < \gamma f'(\rho_R), \\ \rho_R, & \xi \ge \gamma f'(\rho_R), \end{cases} \tag{21}$$

and corresponds to a rarefaction wave. This construction is possible because the characteristic speeds increase from left to right, causing characteristics to spread out rather than intersect. For

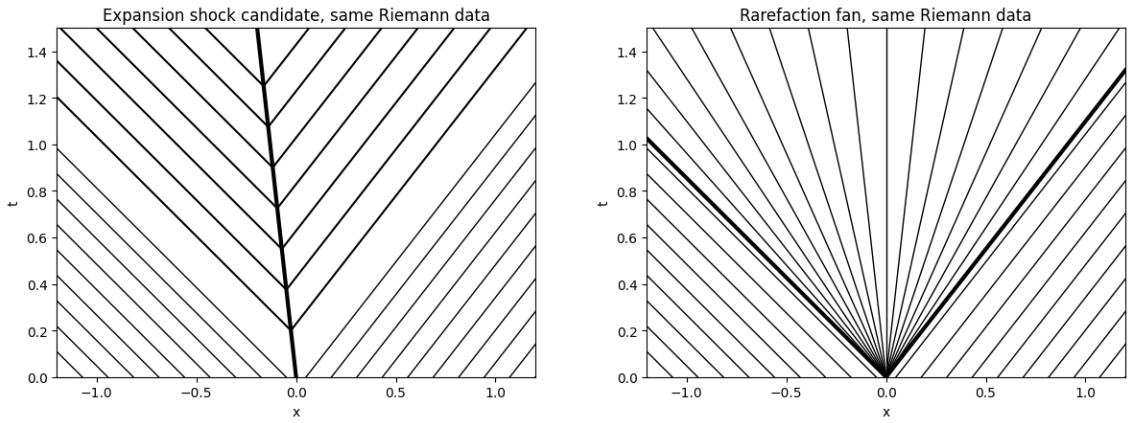

Figure 3: Two different weak solutions of the same Riemann problem (19), $\rho_L = 0.95$ and $\rho_R = 0.15$. Left: a shock solution satisfying the Rankine–Hugoniot condition Section 2.3. Right: a rarefaction wave solution given by (21).

the same values $\rho_L = 0.95$ and $\rho_R = 0.15$, this solution is illustrated in the right panel of Figure 3. It is continuous for $t > 0$ and also satisfies the weak formulation (13).

Both solutions solve the same initial value problem (19) and satisfy the weak solution formulation (13). In particular, both are compatible with the Rankine–Hugoniot condition (18). This shows that weak solutions of (9) are not unique, even when the propagation speed of discontinuities is fixed by the Rankine–Hugoniot condition.

To obtain a unique solution, an additional condition is needed to rule out nonphysical weak solutions. Such a condition is called an *entropy condition*. In this work we use the *Lax entropy condition*, which provides a practical admissibility criterion for shock solutions without developing the full entropy framework.

The Lax entropy condition requires that

$$\gamma f'(\rho_L) > \frac{ds(t)}{dt} > \gamma f'(\rho_R), \tag{22}$$

meaning that characteristics enter the discontinuity from both sides. Equivalently, the shock speed lies strictly between the characteristic speeds of the left and right states. If this inequality is violated, characteristics emerge from the discontinuity and the corresponding weak solution is excluded. In the present example with $\rho_L = 0.95$ and $\rho_R = 0.15$, the shock solution violates (22), while the rarefaction wave satisfies the entropy criterion.

For scalar hyperbolic conservation laws with concave flux, such as (9), imposing the Lax entropy condition guarantees uniqueness of weak solutions and selects the physically relevant solution of the Riemann problem. A rigorous treatment of entropy conditions and the Lax admissibility criterion can be found in [10].

2.5 Junctions and boundary conditions

So far we have only considered traffic flow on a single road. In a real road network, multiple roads meet at junctions. Each individual road is still described by the scaled LWR model (9), but at a junction, additional coupling conditions are required. We consider a road–network consisting of $n+m$ roads connected by a single junction. The set of incoming roads is denoted by $\mathcal{I} = \{1, \dots, n\}$ and the set of outgoing roads by $\mathcal{O} = \{n+1, \dots, n+m\}$. For each road $i \in \mathcal{I} \cup \mathcal{O}$, let $\rho_i(x,t)$ denote the scaled density and let $f_i(\rho_i)$ be the corresponding dimensionless flux. Each road has its own maximum density $\rho_{i,\max}$ and speed limit $v_{i,\max}$.

Using the scaling introduced in (5), the flux on each road must be interpreted in physical units when enforcing conservation at the junction. Conservation of vehicles therefore requires that the

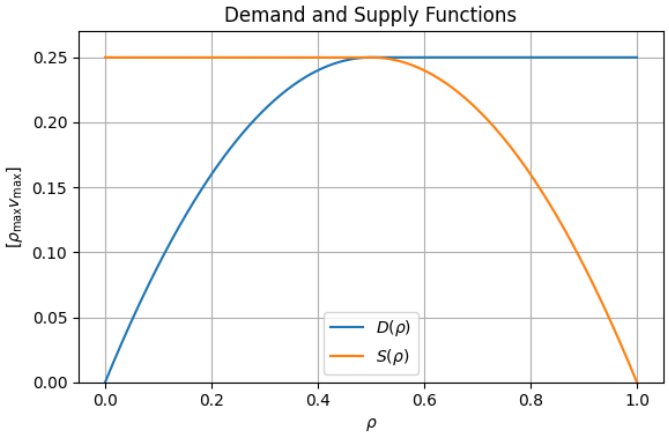

Figure 4: Demand and supply functions.

total physical inflow equals the total physical outflow,

$$\sum_{i\in\mathcal{I}} \rho_{i,\max} v_{i,\max} f_i(1,t) = \sum_{j\in\mathcal{O}} \rho_{j,\max} v_{j,\max} f_j(0,t). \tag{23}$$

This condition enforces mass conservation at the junction, but it does not uniquely determine the individual fluxes. Additional modelling assumptions are therefore required.

We employ a junction rule based on the *demand–supply* principle commonly used for coupling scalar conservation laws on road networks. The underlying framework is the same as that used in [11], where the demand–supply approach is formulated in physical variables and extended to include additional modeling features such as priority rules, merging and yielding behavior, traffic lights (treated as special junctions), and roundabouts (modeled as sequences of junctions). Here, the framework is presented using the scaled variables (5), while demand and supply are interpreted in physical units when enforcing conservation at the junction. We restrict attention to a simplified setting with fixed routing between incoming and outgoing roads in order to obtain a clear baseline description of traffic flow dynamics on road networks. For a thorough, introduction to junction modeling and the demand-–supply principle, starting from the basic 1-to-1 junction, we refer the reader to [11].

For each road i, we introduce a demand function $\bar{D}_i(\rho)$ and a supply function $\bar{S}_i(\rho)$. The demand $\bar{D}_i$ represents the maximal flux that can leave road i, while the supply $\bar{S}_i$ represents the maximal flux that can enter road i. The demand and supply are physical qantities defined by ,

$$\bar{D}_i(\rho) = \begin{cases} \rho_{i,\max} v_{i,\max} f_i(\rho), & \rho \le \sigma_i, \\ \rho_{i,\max} v_{i,\max} f_{i,\max}, & \rho > \sigma_i, \end{cases} \tag{24}$$

$$\bar{S}_i(\rho) = \begin{cases} \rho_{i,\max} v_{i,\max} f_{i,\max}, & \rho \le \sigma_i, \\ \rho_{i,\max} v_{i,\max} f_i(\rho), & \rho > \sigma_i, \end{cases} \tag{25}$$

where σ_i is the density maximizing f_i, and $f_{i,\max} = f_i(\sigma_i)$. For the Greenshields flux (8), the maximum is attained at $\sigma_i = 1/2$ and $f_{i,\max} = f_i(1/2) = 1/4$. For low densities, the demand is equal to the flux, since the number of vehicles limits the outflow, while for higher densities, it saturates at the maximal road capacity $\rho_{i,\max} v_{i,\max} f_{i,\max}$. The supply behaves oppositely: when the density is low, the road can accept maximal inflow, but as congestion builds up for $\rho > \sigma_i$, the admissible inflow decreases and is limited by the flux function itself. This behavior is illustrated in Figure 4.

Traffic from each incoming road is distributed among the outgoing roads according to prescribed turning fractions. We denote the *turning fraction matrix* by

$$A = (\alpha_{ij})_{i\in\mathcal{I},\, j\in\mathcal{O}} \in [0,1]^{|\mathcal{I}|\times|\mathcal{O}|}, \tag{26}$$

the fraction of traffic on incoming road i that is directed into outgoing road j. These coefficients satisfy

$$\sum_{j\in\mathcal{O}} \alpha_{ij} = 1 \qquad \text{for all } i \in \mathcal{I}$$

so that all vehicles leaving an incoming road are distributed among the outgoing roads.

At time t, the demand on incoming road i at the junction is given by $\bar{D}_i(\rho_i(1,t))$, while the supply on outgoing road j is $\bar{S}_j(\rho_j(0,t))$. Accordingly, the portion of the demand from road i directed into road j is defined as

$$\bar{D}_{ij}(t) = \alpha_{ij}\, \bar{D}_i(\rho_i(1,t))\,.$$

This quantity represents the fractional demand from road i to road j before any supply constraints on the outgoing road are enforced.

For each outgoing road j, the total proposed inflow is $\sum_{i\in\mathcal{I}} \bar{D}_{ij}(t)$, which is compared with the available supply $\bar{S}_j(\rho_j(0,t))$. If the total proposed inflow does not exceed the supply, all fluxes are accepted. Otherwise, the incoming contributions are reduced proportionally so that their sum equals the available supply. The resulting (dimensionless) flux from road i to road j is therefore given by

$$f_{ij}(t) = \frac{\bar{D}_{ij}(t)}{\rho_{i,\max} v_{i,\max}} \min\left(1,\ \frac{\bar{S}_j(\rho_j(0,t))}{\sum_{k\in\mathcal{I}} \bar{D}_{kj}(t)}\right). \tag{27}$$

The total flux leaving incoming road $i \in \mathcal{I}$ and entering outgoing road $j \in \mathcal{O}$ is given by

$$f_i(1,t) = \sum_{j\in\mathcal{O}} f_{ij}(t), \tag{28}$$

$$f_j(0,t) = \sum_{i\in\mathcal{I}} f_{ij}(t). \tag{29}$$

We verify that mass is conserved across the junction,

$$\begin{aligned}
\sum_{i\in\mathcal{I}} \rho_{i,\max} v_{i,\max} f_i(1,t) &\overset{(i)}{=} \sum_{j\in\mathcal{O}} \min\left(1,\ \frac{\bar{S}_j(\rho_j(0,t))}{\sum_{k\in\mathcal{I}} \bar{D}_{kj}(t)}\right) \sum_{i\in\mathcal{I}} \bar{D}_{ij}(t) \\
&= \sum_{j\in\mathcal{O}} \min\left(\sum_{i\in\mathcal{I}} \bar{D}_{ij}(t),\ \bar{S}_j(\rho_j(0,t))\right) \\
&\overset{(ii)}{=} \sum_{j\in\mathcal{O}} \rho_{j,\max} v_{j,\max} f_j(0,t).
\end{aligned}$$

In (i), we use (28) together with (27), and factor out the min term, which is independent of i. Step (ii) uses that the total outgoing flux on road j, defined in (29), is given by the minimum of the total proposed demand $\sum_i \bar{D}_{ij}(t)$ and the available supply $\bar{S}_j(\rho_j(0,t))$, which are exactly the two quantities being minimized above. Hence, (23) holds.

The mappings defined by (27), (28) and (29) together define the *junction model* $K : [0,1]^{n+m} \to \mathbb{R}_{\geq 0}^{n+m}$, which maps the vector of boundary densities at the junction to the corresponding vector of boundary fluxes. We write

$$K(\boldsymbol{\rho}_K(t)) = \boldsymbol{f}_K(t), \tag{30}$$

$$\boldsymbol{\rho}_K(t) = \big((\rho_i(1,t))_{i\in\mathcal{I}}, (\rho_j(0,t))_{j\in\mathcal{O}}\big), \tag{31}$$

$$\boldsymbol{f}_K(t) = \big((f_i(1,t))_{i\in\mathcal{I}}, (f_j(0,t))_{j\in\mathcal{O}}\big). \tag{32}$$

All roads in the network are either connected to a junction, in which case the boundary fluxes are given by a junction model such as defined in (30), or they are connected to the network at only one endpoint. The latter case corresponds to external inflow and outflow boundaries.

If the left endpoint $x = 0$ of a road i is not connected to a junction, it is treated as an external inflow boundary with prescribed inflow $f_i^{\text{ext}}(t) \geq 0$, scaled with its own parameters $\rho_{i,\max}^{\text{ext}}$ and $v_{i,\max}^{\text{ext}}$. To model limited access to the network, we introduce a queue length $l_i(t) \geq 0$ at the boundary. The corresponding external demand is defined by

$$\bar{D}_i^{\text{ext}}(l_i, t) = \begin{cases} \rho_{i,\max}^{\text{ext}} v_{i,\max}^{\text{ext}} f_i^{\text{ext}}(t), & l_i(t) = 0, \\ \rho_{i,\max} v_{i,\max} f_{i,\max}, & l_i(t) > 0. \end{cases}$$

The inflow is restricted by the available supply of the road, and the resulting boundary flux is given by

$$f_i(0,t) = \frac{1}{\rho_{i,\max} v_{i,\max}} \min\big(\bar{D}_i^{\text{ext}}(l_i, t),\ \bar{S}_i(\rho_i(0,t))\big)\,. \tag{33}$$

The queue length evolves according to

$$\frac{d\bar{l}_i(t)}{dt} = \rho_{i,\max}^{\text{ext}} v_{i,\max}^{\text{ext}} f_i^{\text{ext}}(t) - \rho_{i,\max} v_{i,\max} f_i(0,t),$$

which guarantees $\bar{l}_i(t) \geq 0$ and increases exactly when the external inflow exceeds what can enter the road.

If the right endpoint $x = 1$ of a road i is not connected to a junction, it is treated as an external outflow boundary. We assume free outflow and set

$$f_i(1,t) = \frac{\bar{D}_i(\rho_i(1,t))}{\rho_{i,\max} v_{i,\max}}. \tag{34}$$

The boundary fluxes defined in (33) and (34), together with the junction model (30), provide the boundary conditions that couple the single–road model (9) into a road network. In other words, these rules specify how cars enter and leave the system and how traffic is transferred between different roads.

2.6 Road–network model with a single junction

We consider a class of road networks consisting of $n + m$ roads connected by a single junction. The set of all roads is denoted by $\mathcal{R} = \{1, \ldots, n+m\}$, with incoming roads $\mathcal{I} = \{1, \ldots, n\}$ and outgoing roads $\mathcal{O} = \{n+1, \ldots, n+m\}$. On each road $r \in \mathcal{R}$, we consider the scaled LWR model given in (9). That is, for every $r \in \mathcal{R}$ the density ρ_r satisfies

$$\begin{cases} (\rho_r)_t + \gamma_r\big(1 - 2\rho_r\big)(\rho_r)_x = 0, & (x,t) \in (0,1) \times \mathbb{R}_+, \\ \rho_r(x,0) = \rho_{r,0}(x), & x \in (0,1), \end{cases} \tag{35}$$

with $\gamma_r = \frac{v_{r,\max}}{b_r L}$. Each road has its own parameters $v_{r,\max}$, $\rho_{r,\max}$, and b_r, while all roads share the same network scaling.

For roads $r \in \mathcal{I}$, the boundary at $x = 0$ is treated as an external boundary, and for roads $r \in \mathcal{O}$, the boundary at $x = 1$ is treated as an external boundary. These external boundaries are prescribed by the external inflow and free outflow conditions, (33) and (34).

The coupling at the junction of the road network is given by a junction model $K : [0,1]^{n+m} \to \mathbb{R}_{\geq 0}^{n+m}$, which maps the vector of boundary densities at the junction to the corresponding vector of boundary fluxes. The model K is assumed to be continuous and to satisfy the mass conservation law (23). In this report, we only consider junction models that depend on the current boundary densities. This is the same type of junction model as the one used in the numerical experiments, see (30).

The formulation above describes a general class of road networks with one junction. In general, one may consider larger road networks with several junctions. In this report, however, all numerical experiments are carried out on road networks consisting of a single junction.

2.7 Alternative traffic flow models

So far, we have only considered the LWR model, where the velocity is assumed to be entirely determined by the density through a relation of the form (2). In this case, the flux depends only on ρ, and the model has a single unknown. We have demonstrated that this gives a simple scalar hyperbolic conservation law that already captures important features of traffic flow, such as the formation of shocks and rarefaction waves.

If we do not want to assume that the velocity is completely determined by the density, we can instead let the velocity be an additional unknown and add a separate equation for it. This leads to a model with two unknowns, $\bar{\rho}$ and $\bar{v}$. A well-known example is the *Aw–Rascle–Zhang* (ARZ) model [12, 13], which in physical variables can be expressed as

$$\begin{cases} \bar{\rho}_t + (\bar{\rho}\bar{v})_x = 0, \\ [\bar{\rho}(\bar{v} + p(\bar{\rho}))]_t + [\bar{\rho}\bar{v}(\bar{v} + p(\bar{\rho}))]_x = 0, \end{cases} \tag{36}$$

where $p(\bar{\rho})$ is a pressure function. The first equation is the same mass conservation law as in the LWR model. The second equation expresses conservation of momentum and models how drivers adapt their speed in response to the local traffic conditions. If one applies the same scaling as in (5), the factor γ would appear in front of the spatial derivatives, just as in (9). Since we only use (36) for illustration and do not simulate this model, we keep it in this physical form.

Models of the type (36) can reproduce more complex effects than LWR, for example stop–and–go waves and different acceleration behaviour. On the other hand, they are more complicated, require more parameters, and are more expensive to solve numerically. The aim of this work is to analyse the LWR model and its behaviour at junctions, with a focus on the associated numerical methods, rather than to reproduce all aspects of real traffic.

Even within the LWR framework there are many possible choices for the velocity–density relation $v(\rho)$ and the corresponding flux $f(\rho) = \rho v(\rho)$. In this work, we use the Greenshields relation given in (7). This is the simplest choice, since it assumes a linear decrease of velocity with density and leads to a quadratic flux.

Several alternative velocity laws have been proposed in the literature. In dimensionless form, the *Greenberg* model [14] uses the relation $v(\rho) = \ln(1/\rho)$, which reflects that drivers react strongly to density changes in light traffic. The *Underwood* model [15] assumes an exponential decay, $v(\rho) = \exp(-\rho)$, giving a smooth and gradual reduction of speed as the density increases. The *Newell* model [16] uses the form $v(\rho) = 1 - \exp(-1/\rho + 1)$, providing an even smoother transition between free flow and congestion. Each of these relations produces a different flux function $f(\rho)$ and therefore different traffic behaviour, even though the governing conservation law still has the same form as in (6).

3 Finite volume methods

The main challenge in the numerical solution of scalar conservation laws is the possible appearance of discontinuities. As shown in Sections 2.2 and 2.3, even smooth initial data may lead to the formation of shocks in finite time, after which the solution is no longer differentiable. Classical finite–difference methods approximate differential operators by Taylor expansions, which require the solution to be sufficiently smooth. Since solutions of the LWR model and other hyperbolic conservation laws generally contain discontinuities, the classical derivative is not defined everywhere and finite–difference methods therefore lack a proper mathematical justification at shocks.

Instead, we use *finite volume* methods. These methods are derived directly from the integral form of the conservation law and therefore remain valid even when the solution contains discontinuities. While finite–difference and finite volume schemes may in some cases lead to identical update formulas, the crucial distinction lies in how the discrete quantities are interpreted. Rather than approximating pointwise values of the solution, finite volume methods approximate *cell averages*

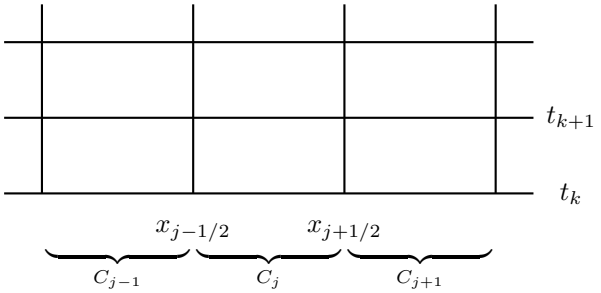

Figure 5: The grid for the finite volume methods.

of the conserved quantity over small control volumes. The numerical scheme is constructed by balancing the fluxes entering and leaving each cell, so that conservation is satisfied exactly at the discrete level. When combined with appropriate numerical fluxes, finite volume methods converge to the unique entropy solution, which represents the physically relevant traffic flow described in Section 2.3. For these reasons, finite volume methods are the standard choice for hyperbolic conservation laws with shocks and will be used throughout this work. General introductions to finite volume methods for hyperbolic conservation laws can be found in [17, 10].

3.1 Discretization and the iteration scheme

The first step in any numerical approximation is to discretize the domain into a grid. Recall that on a single road, we work on the dimensionless interval $x \in [0, 1]$, which corresponds to a physical road of length L. We divide the road into $N+1$ equidistant grid points, such that $\Delta x = 1/(N)$. The grid points are given by $x_j = j\Delta x$ for $j = 0, \dots, N$. We also define the midpoints $x_{j\pm1/2} = x_j \pm \Delta x/2$.

The first step in any numerical approximation is to discretize the domain into a grid. Recall that we work on the dimensionless interval $x \in [0, 1]$, which corresponds to a physical road of length L. We divide the interval into N uniform control volumes of width $\Delta x = 1/N$. The cell interfaces are located at $x_{j+1/2} = j\Delta x$ for $j = 0, \dots, N$, while the cell centers are given by $x_j = (j - \frac{1}{2})\Delta x$ for $j = 1, \dots, N$.

The control volumes (cells) are defined as $C_j = [x_{j-1/2},\, x_{j+1/2})$ for $j = 1, \dots, N$. At the boundaries, this corresponds to the first and last cells $C_1 = [0,\, x_{3/2})$ and $C_N = [x_{N-1/2},\, 1]$. In contrast to the fixed spatial step, the time step will be allowed to vary and will later be chosen according to a suitable stability condition. The grid is visualized in Figure 5.

Rather than approximating pointwise values of the solution, the numerical scheme is formulated in terms of cell averages over the control volumes. For an internal control volume C_j, the exact cell average of the density at time t is defined by

$$\rho_j(t) = \frac{1}{\Delta x} \int_{C_j} \rho(x, t)\, dx. \tag{37}$$

We now derive the finite volume formulation of the dimensionless LWR equation (6) on a single control volume. Representing the scaled LWR model in integral form as in (1) and applying it to the cell C_j, we obtain the exact conservation law

$$\frac{d}{dt} \int_{x_{j-1/2}}^{x_{j+1/2}} \rho(x, t)\, dx = -\gamma\big(f(\rho(x_{j+1/2}, t)) - f(\rho(x_{j-1/2}, t))\big). \tag{38}$$

Dividing (38) by Δx and using the definition of the cell average (37) gives the exact *semi-discrete* finite volume formulation

$$\dot{\rho}_j(t) = -\frac{\gamma}{\Delta x}\big(f(\rho(x_{j+1/2}, t)) - f(\rho(x_{j-1/2}, t))\big). \tag{39}$$

This is a coupled system of ODEs for the exact cell averages, with one ODE for each control volume. The system is discrete in space but still continuous in time. To obtain a fully discrete scheme, we integrate the semi-discrete system (39) from t_k to t_{k+1} and use the fundamental theorem of calculus in time. Introducing the time step $\Delta t_k = t_{k+1} - t_k$ and the notation $\rho_j^k \approx \rho_j(t_k)$, this leads to the finite volume update

$$\rho_j^{k+1} = \rho_j^k - \frac{\gamma\,\Delta t_k}{\Delta x}\big(F_{j+1/2}^k - F_{j-1/2}^k\big), \tag{40}$$

$$F_{j+1/2}^k \approx \frac{1}{\Delta t_k}\int_{t_k}^{t_{k+1}} f(\rho(x_{j+1/2},t))\,dt. \tag{41}$$

Here $F_{j+1/2}^k$ denotes the numerical flux through the interface $x_{j+1/2}$ during the time step $[t_k, t_{k+1}]$.

If the numerical fluxes $F_{j\pm1/2}^k$ are chosen as functions of the current approximated cell averages $\{\rho_j^k\}_j$, then the update rule (40) defines a fully explicit and closed scheme. The new values ρ_j^{k+1} depend only on the solution at the previous time level. The initial cell averages are obtained from the initial density as

$$\rho_j^0 = \frac{1}{\Delta x}\int_{C_j} \rho_0(x)\,dx. \tag{42}$$

The update formula (40) expresses the conservation of mass in each control volume. The only way the amount of mass in a cell can change from one time step to the next is through the fluxes across its two boundaries. Different first-order finite volume schemes differ only in how the numerical fluxes $F_{j\pm1/2}^k$ are approximated.

3.2 Godunov flux and time stepping rule

The general finite volume update formula (40) defines a fully discrete scheme once the numerical fluxes $F_{j\pm1/2}^k$ have been specified. Godunov's idea [18] was to use the exact solution of the underlying PDE at each cell interface to define the numerical flux, leading to the Godunov finite volume method. This directly links the numerical method to the theory of characteristics, weak solutions, entropy conditions and shocks developed in Sections 2.2 and 2.3.

At time t_k the finite volume approximation is piecewise constant, $\rho(x,t_k) \approx \rho_j^k$ on each control volume C_j, as illustrated in Figure 6. Consequently, each interface $x_{j+1/2}$ defines a local Riemann problem for the LWR model (9),

$$\begin{cases} \rho_t + \gamma(1-2\rho)\rho_x = 0, & t > t_k, \\ \rho(x,t_k) = \begin{cases} \rho_j^k, & x < x_{j+1/2}, \\ \rho_{j+1}^k, & x > x_{j+1/2}. \end{cases} \end{cases} \tag{43}$$

As discussed in Section 2.4, the Riemann problem generally admits several weak solutions. For the scaled LWR model with concave Greenshields flux (9), imposing the Lax entropy condition (22) selects the unique physically relevant solution, which consists of either a single shock or a rarefaction wave. As long as characteristics from neighbouring cells do not reach the interface $x = x_{j+1/2}$ during a single time step, the entropy solution at the interface is constant in time and depends only on the left and right states ρ_j^k and ρ_{j+1}^k. In this case,

$$\rho(x_{j+1/2},t) = \rho_{\text{int}}(\rho_j^k, \rho_{j+1}^k), \qquad t \in (t_k, t_{k+1}),$$

where $\rho_{\text{int}}(\rho_j^k, \rho_{j+1}^k)$ denotes the value of the entropy solution of the local Riemann problem at the interface. The exact flux integral (41) therefore reduces to

$$F_{j+1/2}^k = \frac{1}{\Delta t_k}\int_{t_k}^{t_{k+1}} f(\rho(x_{j+1/2},t))\,dt = f\big(\rho_{\text{int}}(\rho_j^k, \rho_{j+1}^k)\big). \tag{44}$$

Finally, for the local Riemann problems to remain independent during one time step, waves must not travel more than one cell width before the next update. The fastest propagation speed in

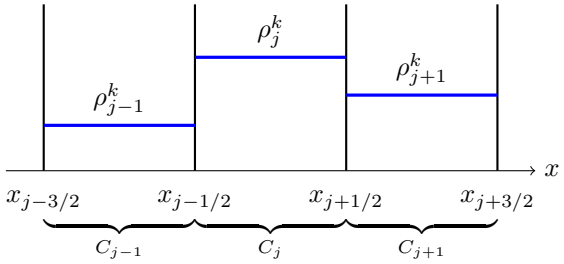

Figure 6: Finite volume solution at time t_k represented by piecewise constant cell averages ρ_j^k over the cells C_{j-1}, C_j, C_{j+1}. At each interface $x_{j+1/2}$ the jump between neighbouring cell values defines a local Riemann problem for the LWR model.

the numerical solution is given by the maximum characteristic speed, (11). Requiring that no characteristic travels further than one cell width Δx over one time step Δt_k therefore yields the stability restriction $\max_j |\gamma f'(\rho_j^k)|\, \Delta t_k \leq \Delta x$. Equivalently, this can be written in the standard *Courant–Friedrichs–Lewy condition* (CFL condition) form

$$\Delta t_k \leq \mathrm{CFL}\, \frac{\Delta x}{\gamma \max_j |1 - 2\rho_j^k|}, \qquad 0 < \mathrm{CFL} \leq 1. \tag{45}$$

3.3 Rusanov flux

The Godunov flux introduced in Section 3.2 is defined by evaluating the exact entropy solution of the local Riemann problem (43) at each cell interface. For the scaled LWR model (9), the associated Riemann problem admits a simple solution consisting of either a single shock or a rarefaction wave, so the Godunov flux can be evaluated explicitly. More generally, however, the evaluation of the exact Riemann solution may be expensive or impractical for more complicated flux functions or models. This motivates the use of *approximate Riemann solvers*, where the exact wave structure is replaced by a simpler approximation that still respects conservation.

We again consider the Riemann problem at the interface $x_{j+1/2}$ defined by the piecewise constant finite volume solution at time t_k, see (43). Instead of resolving the exact solution, we approximate it by two waves, one propagating to the left with speed $s_{j+1/2}^l < 0$ and the other propagating to the right with speed $s_{j+1/2}^r > 0$, separated by a constant intermediate state $\rho_{j+1/2}^*$.

Introducing the intermediate interface flux $f_{j+1/2}^*$, which represents the value of the scalar flux $f(\rho)$ at $x_{j+1/2}$ in the approximate Riemann solution, and imposing conservation across each wave via the Rankine–Hugoniot condition (18), we obtain the system

$$f(\rho_{j+1}^k) - f_{j+1/2}^* = s_{j+1/2}^r\big(\rho_{j+1}^k - \rho_{j+1/2}^*\big), \qquad f_{j+1/2}^* - f(\rho_j^k) = s_{j+1/2}^l\big(\rho_{j+1/2}^* - \rho_j^k\big). \tag{46}$$

Solving this system for $f_{j+1/2}^*$ yields the numerical flux

$$F_{j+1/2}^k = \frac{s_{j+1/2}^r f(\rho_j^k) - s_{j+1/2}^l f(\rho_{j+1}^k) + s_{j+1/2}^r s_{j+1/2}^l \big(\rho_{j+1}^k - \rho_j^k\big)}{s_{j+1/2}^r - s_{j+1/2}^l}, \tag{47}$$

which is constant in time over the interval (t_k, t_{k+1}) and can therefore be used directly in the finite volume update (40).

A commonly used simplification is to choose symmetric wave speeds $s_{j+1/2}^r = -s_{j+1/2}^l = s_{j+1/2}$, in which case (47) reduces to

$$F_{j+1/2}^k = \frac{1}{2}\Big(f(\rho_j^k) + f(\rho_{j+1}^k)\Big) - \frac{s_{j+1/2}}{2}\big(\rho_{j+1}^k - \rho_j^k\big). \tag{48}$$

Different numerical schemes are obtained by different choices of the wave speed $s_{j+1/2}$. The classical *Lax–Friedrichs* scheme [19] uses a global upper bound determined by the time step, which leads to a comparatively diffusive method.

A less diffusive alternative is obtained by selecting the wave speed locally, based on the characteristic speeds of the conservation law. This choice leads to the *Rusanov flux* [20], also known as the local Lax–Friedrichs flux,

$$F_{j+1/2}^k = \frac{1}{2}\Big(f(\rho_j^k) + f(\rho_{j+1}^k)\Big) - \frac{1}{2}\max\Big(|f'(\rho_j^k)|,\, |f'(\rho_{j+1}^k)|\Big)\big(\rho_{j+1}^k - \rho_j^k\big). \tag{49}$$

For the LWR model, where $f(\rho) = \rho(1-\rho)$, we have $f'(\rho) = 1 - 2\rho$. The physical characteristic speed of the conservation law $\rho_t + (\gamma f(\rho))_x = 0$ is therefore $\gamma f'(\rho) = \gamma(1-2\rho)$, which enters the CFL condition through the prefactor γ in the finite volume update. As a result, the Rusanov flux depends only on the local cell values.

Compared to the Lax–Friedrichs scheme, the Rusanov scheme leads to a considerable improvement in accuracy while retaining a simple and robust structure, as demonstrated by numerical experiments [17]. The Rusanov scheme is advanced in time using the same CFL restriction (45).

3.4 Consistency, stability and convergence of finite volume methods

A numerical scheme for a conservation law should satisfy a small set of basic structural properties in order to approximate the physically relevant entropy solution. In this section we briefly summarize these properties for the finite volume schemes considered in this work, without attempting a full theoretical analysis.

A numerical flux is said to be *consistent* with the physical flux if it reproduces the exact flux for constant states. More precisely, for any constant density $\bar{\rho}$ the numerical flux must satisfy

$$F(\bar{\rho}, \bar{\rho}) = f(\bar{\rho}).$$

This ensures that constant solutions of the conservation law are preserved exactly by the numerical scheme. Both the Godunov flux Section 3.2 and the Rusanov flux (49) are consistent in this sense, since they reduce to the physical flux whenever the left and right states coincide.

finite volume schemes are *conservative* by construction. The update formula (40) expresses the change in the cell average ρ_j^k solely in terms of fluxes through the cell interfaces. As a result, the total discrete mass is conserved exactly, up to boundary effects. This property mirrors the integral conservation law (38) and is essential for the correct propagation of shocks and other discontinuities.

Stability of the explicit schemes considered here is ensured by imposing a CFL condition. The CFL restriction (45) guarantees that information does not propagate across more than one cell during a single time step. Under this condition, both the Godunov and Rusanov schemes remain stable and produce bounded numerical solutions.

For scalar conservation laws, a classical result due to Lax and Wendroff states that any conservative, consistent and stable scheme converges, as the mesh is refined, to a weak solution of the conservation law. When combined with a monotone or entropy-stable numerical flux, the limit solution is the unique entropy solution. In particular, first-order finite volume schemes with monotone fluxes, such as the Godunov and Rusanov schemes, are known to converge to the physically correct solution of the LWR model, see for example, [10].

3.5 Higher-order methods

The first-order finite volume schemes introduced in Sections 3.2 and 3.3 are robust and converge to the entropy solution of the LWR model. However, they are also known to be significantly diffusive. In particular, shocks are smeared over several cells, and smooth solution features are damped as the simulation progresses. This loss of accuracy is an inherent limitation of first-order methods and motivates the use of higher-order schemes.

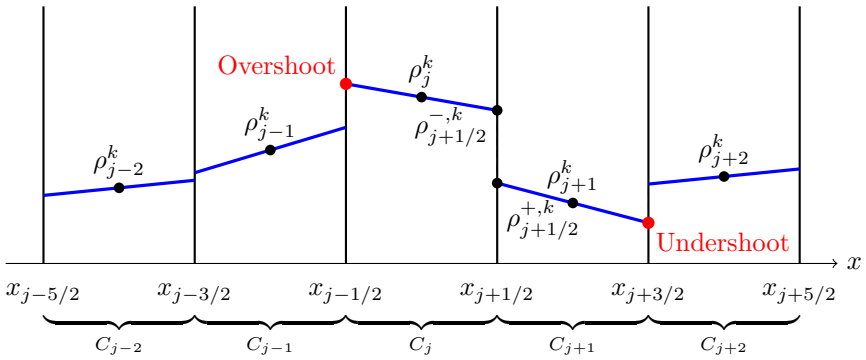

Figure 7: Unrestricted piecewise linear reconstruction using centred slopes. The reconstructed interface values may overshoot or undershoot near sharp changes in the cell averages, which can lead to oscillations after the flux update.

To increase accuracy, we consider second-order extensions based on a combination of spatial reconstruction and higher-order time integration. The main challenge is to improve accuracy in smooth regions while maintaining stability and non-oscillatory behaviour near discontinuities.

3.5.1 Spatial reconstruction

At a fixed time level t_k, a first-order finite volume method represents the numerical solution through the cell averages $\{\rho_j^k\}_j$. In space, this corresponds to a piecewise constant approximation, as illustrated in Figure 6. This representation is robust, but it is also the main source of numerical diffusion. Since no spatial variation is resolved inside each cell, smooth gradients are gradually smeared out as the simulation evolves.

Higher-order accuracy in space is obtained by keeping the same cell averages, but reconstructing a more detailed spatial profile inside each cell. We therefore replace the piecewise constant reconstruction by a piecewise linear one. On each cell C_j, we define

$$\rho_j(x) = \rho_j^k + \sigma_j^k(x - x_j), \qquad x \in C_j, \tag{50}$$

where σ_j^k is a numerical slope. This reconstruction is conservative by construction, since the cell average of $\rho_j(x)$ equals ρ_j^k.

A natural choice for the slope is the centered difference

$$\sigma_j^k = \frac{\rho_{j+1}^k - \rho_{j-1}^k}{2\Delta x}, \tag{51}$$

which is second-order accurate when the solution is smooth. However, near shocks or sharp gradients this unrestricted choice may produce overly steep slopes. As a result, the reconstructed values at cell interfaces can overshoot or undershoot the neighbouring cell averages, as shown in Figure 7. These effects may lead to nonphysical oscillations after the flux update.

To prevent such oscillations, the slopes are limited. The construction of slope limiters is guided by the concept of total variation. For a discrete sequence $\{\rho_j\}_j$, the total variation is defined as

$$\mathrm{TV}(\rho) = \sum_j |\rho_j - \rho_{j-1}|. \tag{52}$$

Spurious oscillations typically increase the total variation. A numerical scheme is therefore called total-variation-diminishing if

$$\mathrm{TV}(\rho^{k+1}) \le \mathrm{TV}(\rho^k), \tag{53}$$

which ensures that new oscillations are not created as the solution evolves.

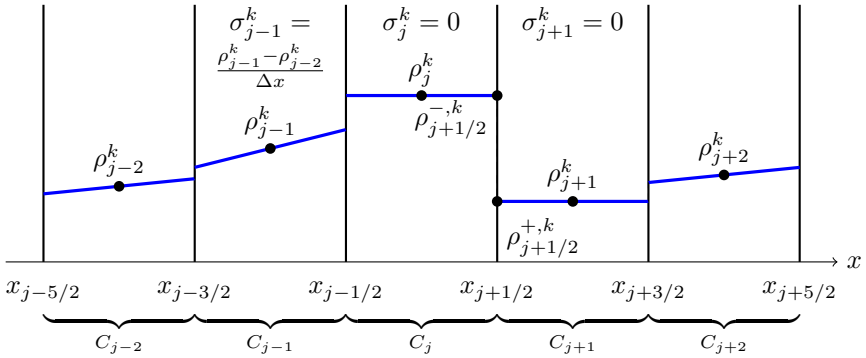

Figure 8: Slope limited piecewise linear reconstruction using the minmod limiter. In cells where the one-sided differences have opposite signs, the limiter sets the slope to zero. This removes the overshoot and undershoot seen in Figure 7 and reduces the risk of oscillations after the flux update.

In this work, we use the minmod limiter,

$$\sigma_j^k = \operatorname{minmod}\left(\frac{\rho_j^k - \rho_{j-1}^k}{\Delta x}, \frac{\rho_{j+1}^k - \rho_j^k}{\Delta x}\right), \tag{54}$$

where

$$\operatorname{minmod}(a,b) = \begin{cases} \operatorname{sign}(a)\min(|a|,|b|), & \text{if } \operatorname{sign}(a) = \operatorname{sign}(b), \\ 0, & \text{otherwise.} \end{cases} \tag{55}$$

When the solution varies smoothly across neighbouring cells, the limiter retains a nonzero slope and the reconstruction remains close to linear. Near local extrema or discontinuities, the limiter reduces the slope or sets it to zero. In that case, the reconstruction on the affected cell becomes locally constant and the method effectively reduces to the first-order finite volume scheme. This mechanism is essential for stability near shocks and is illustrated in Figure 8.

The reconstructed values at the cell interfaces are obtained by evaluating the linear functions at the cell boundaries. At the interface $x_{j+1/2}$ we define

$$\rho_{j+1/2}^{-,k} = \rho_j^k + \frac{\Delta x}{2}\sigma_j^k, \qquad \rho_{j+1/2}^{+,k} = \rho_{j+1}^k - \frac{\Delta x}{2}\sigma_{j+1}^k. \tag{56}$$

These left and right interface values generally differ, which gives rise to a local Riemann problem at each cell boundary. The numerical flux is computed by solving this Riemann problem exactly or approximately using the same flux functions as in the first-order method, such as the Godunov (**??**) or Rusanov (**??**), but now evaluated with the reconstructed states. The numerical flux at the interface $x_{j+1/2}$ is therefore given by $F_{j+1/2}^k = F\big(\rho_{j+1/2}^{-,k}, \rho_{j+1/2}^{+,k}\big)$, where $F(\cdot,\cdot)$ denotes a consistent numerical flux. Compared to the first-order finite volume method, the overall structure of the scheme is unchanged. The only difference is that the flux is now computed from the reconstructed interface values rather than from the cell averages. This is what yields higher spatial accuracy in smooth regions.

3.5.2 Time integration

As shown in (39), spatial discretization of the conservation law leads to a system of ordinary differential equations for the cell averages $\rho_j(t)$. A fully discrete finite volume scheme is obtained by applying a time-stepping method to this semi-discrete system. In particular, the first-order schemesintroduced earlier are obtained by advancing the solution in time using a single explicit time step.

A common approach is to use explicit *Runge–Kutta* (RK) methods. These methods advance the solution in time by evaluating the right-hand side of the semi-discrete finite volume scheme at one

or more intermediate stages. The simplest such method is the *first-order explicit RK method*, also known as the *forward Euler method*.

Using forward Euler, the update of the cell averages takes the form

$$\rho_j^{k+1} = \rho_j^k - \frac{\gamma\,\Delta t_k}{\Delta x}\big(F_{j+1/2}^k - F_{j-1/2}^k\big).$$

This is precisely the update rule used in the first-order finite volume schemes introduced earlier. Once the numerical flux is specified, the method is fully defined and first-order accurate in time.

To obtain second–order accuracy in time, we replace the forward Euler step by a second-order RK method. In this work, we use a *strong-stability-preserving* second-order RK method (SSPRK2). Given the solution ρ_j^k at time t_k, the method is defined by

$$\rho_j^{(1)} = \rho_j^k - \frac{\gamma\,\Delta t_k}{\Delta x}\big(F_{j+1/2}^k - F_{j-1/2}^k\big), \tag{57}$$

$$\rho_j^{k+1} = \frac{1}{2}\rho_j^k + \frac{1}{2}\left(\rho_j^{(1)} - \frac{\gamma\,\Delta t_k}{\Delta x}\big(F_{j+1/2}^{(1)} - F_{j-1/2}^{(1)}\big)\right), \tag{58}$$

where $F_{j\pm1/2}^{(1)}$ denotes the numerical flux evaluated using the intermediate stage values $\rho^{(1)}$, i.e. the same flux function as in the first stage applied to the reconstructed interface states obtained from $\rho^{(1)}$.

The SSPRK2 method can be interpreted as a convex combination of forward Euler steps and therefore inherits the stability properties of the first-order scheme, provided the time step satisfies the CFL condition (45).

3.5.3 Second-order finite volume scheme

Combining slope-limited spatial reconstruction with SSPRK2 time integration yields a second-order accurate finite volume scheme. In smooth regions, this means that the numerical error decreases quadratically as the grid is refined, with second-order accuracy in both space and time. Near discontinuities, the slope limiter reduces the reconstruction locally, ensuring a stable and non-oscillatory approximation.

We note that even higher-order finite volume methods can be constructed by using higher-degree polynomial reconstructions in space together with higher-order Runge–Kutta time integration. While such methods can further reduce numerical diffusion in smooth regions, they require more sophisticated limiters and are more sensitive to stability constraints.

3.6 Boundary conditions for road–networks

To complete the finite volume discretization of the road–network model Section 2.6, we specify how boundary conditions are imposed at external road endpoints and at junction interfaces. As established in Section 2.5, all couplings are prescribed through flux conditions. Numerically, this means that the boundary condition decides the numerical flux at the boundary interface.

Recall that each road $r \in \mathcal{R}$ has physical length L_r and is discretized into N_r finite volume cells. For numerical convenience, we introduce a common base length $L > 0$, which represents the length of a single spatial building block used throughout the network. Each road is then composed of an integer number of such blocks, and we write

$$L_r = b_r L, \qquad N_r = b_r N,$$

where $b_r \in \mathbb{N}$ is a road-specific scaling factor and N is a global resolution parameter, giving the number of finite volume cells per block.

This construction implies that all roads are composed of integer multiples of a reference segment of length L, and that each such segment is discretized using the same number N of cells. As a

consequence, the spatial discretization aligns exactly with the physical length of each road, and all finite volume cells across the network have the same physical length, $\Delta\bar{x}_r = (b_r L)/(b_r N) = L/N$. The corresponding scaled velocity parameter satisfies $\gamma_r = v_{r,\max}/(b_r L)$.

We denote the discrete traffic state by

$$\{\rho_{r,j}^k\}_{r\in\mathcal{R},\, j\in\mathcal{X}_r,\, k\in\mathcal{T}},$$

where $\mathcal{R}$ is the set of roads, $\mathcal{X}_r = \{1, 2, \ldots, b_r N\}$ is the set of finite volume cells on road r, and $\mathcal{T} = \{0, 1, \ldots, M\}$ denotes the discrete time levels. Each road r has exactly one cell adjacent to the junction, indexed by

$$j_r^J = \begin{cases} b_r N, & r \in \mathcal{I}, \\ 1, & r \in \mathcal{O}. \end{cases}$$

We collect these cells in the set $\mathcal{K} = \{(r, j_r^J) \mid r \in \mathcal{R}\}$.

At each time level t_k, the junction density vector is approximated by the junction-adjacent cell averages

$$\boldsymbol{\rho}_K^k = \big((\rho_{r,j_r^J}^k)_{r\in\mathcal{I}}, (\rho_{r,j_r^J}^k)_{r\in\mathcal{O}}\big),$$

which is the numerical counterpart of (31). In the finite volume update, the numerical flux at the interface adjacent to the junction is replaced by the corresponding component of K, which acts as a boundary condition at the junction endpoints,

$$F_{r,j_r^J+1/2}^k = K(\boldsymbol{\rho}_K^k)_r \quad \text{for } r \in \mathcal{I}, \qquad F_{r,j_r^J-1/2}^k = K(\boldsymbol{\rho}_K^k)_r \quad \text{for } r \in \mathcal{O}. \tag{59}$$

Thus, the junction rule directly prescribes the numerical fluxes at the junction interfaces, while the interior finite volume scheme remains unchanged.

Next, we define the numerical fluxes at external boundaries, which serve as the discrete counterparts of the continuous boundary conditions, see (33) and (34). The main difference from the continuous formulation is that all quantities are updated over finite time steps, which requires additional care to ensure physical admissibility.

If a road r is not connected to a junction at its left endpoint $x = 0$, an external inflow $f_r^{\text{ext},k} \geq 0$ is prescribed at time t_k. As in the continuous model, a queue of physical length $\bar{l}_r^k \geq 0$ is stored at the boundary to model limited access to the network. Over the time step $[t_k, t_{k+1}]$, the admissible physical inflow demand is defined as

$$\bar{D}_r^{\text{ext},k} = \min\left(\rho_{r,\max} v_{r,\max} f_{r,\max},\ \rho_{r,\max}^{\text{ext}} v_{r,\max}^{\text{ext}} f_r^{\text{ext},k} + \frac{\bar{l}_r^k}{\Delta t}\right).$$

The resulting boundary flux is then obtained using the same demand–supply principle as in the junction coupling,

$$f_r^{\text{in},k} = \frac{1}{\rho_{r,\max} v_{r,\max}} \min\big(\bar{D}_r^{\text{ext},k},\ \bar{S}_r\big(\rho_{r,0}^k\big)\big). \tag{60}$$

Consequently the queue length update becomes,

$$\bar{l}_r^{k+1} = \bar{l}_r^k + \Delta t_k\, \rho_{r,\max}^{\text{ext}} v_{r,\max}^{\text{ext}} f_r^{\text{ext},k} - \Delta t_k\, \rho_{r,\max} v_{r,\max} f_r^{\text{in},k},$$

and is truncated if necessary to ensure that $\bar{l}_r^{k+1} \geq 0$.

If a road r is not connected to a junction at its right endpoint $x = 1$, a free outflow condition is imposed by taking the boundary flux equal to the road demand,

$$f_r^{\text{out},k} = \frac{\bar{D}_r\big(\rho_{r,N_r}^k\big)}{\rho_{r,\max} v_{r,\max}}. \tag{61}$$

Similarly, the external boundary numerical fluxes become,

$$F_{r,1/2}^k = f_r^{\text{in},k} \quad \text{for } r \in \mathcal{I}, \qquad F_{r,b_r N+1/2}^k = f_r^{\text{out},k} \quad \text{for } r \in \mathcal{O}. \tag{62}$$

In the implementation, the external fluxes and the junction fluxes are not inserted directly into the finite volume update formula. Instead, they are imposed through *ghost* cells so that the numerical flux computed by the chosen Riemann solver coincides with the prescribed physical flux.

For a first-order scheme, one ghost cell is introduced at each road endpoint. The boundary-adjacent interior cells are $\rho^k_{r,1}$ at the left endpoint and ρ^k_{r,N_r} at the right endpoint. Given a prescribed boundary or junction flux at the left or right endpoint, denoted by $f^k_{r,1/2}$ and $f^k_{r,N_r+1/2}$ and defined by (59) and (62), the ghost cell value is chosen such that

$$F\big(\rho^k_{r,0}, \rho^k_{r,1}\big) = f^k_{r,1/2}, \qquad \text{or} \qquad F\big(\rho^k_{r,N_r}, \rho^k_{r,N_r+1}\big) = f^k_{r,N_r+1/2},$$

respectively.

For second-order schemes, additional ghost cells are introduced as required by the reconstruction procedure. Their values are set consistently so that the reconstructed interface flux equals the prescribed boundary or junction flux, denoted by $f^k_{r,1/2}$ or $f^k_{r,N_r+1/2}$. Apart from this reconstruction step, boundary and junction conditions are treated in the same way as in the first-order case.

In addition to the interior CFL restriction (45), boundary and junction interfaces are subject to an additional constraint. At such interfaces, the numerical flux is prescribed by a boundary condition or a junction rule and generates waves whose characteristic speed depends on the imposed flux value.

For a road r with prescribed boundary or junction flux f^k_r, defined by Equations (60) to (62), the corresponding CFL condition reads

$$\Delta t_k \le \mathrm{CFL}\, \frac{\Delta x_r}{\gamma_r |1 - 4 f^k_r|}.$$

The global time step is therefore chosen as

$$\Delta t_k = \min_{r \in \mathcal{R}} \left\{ \mathrm{CFL}\, \frac{\Delta x_r}{\gamma_r \max_j |1 - 2\rho^k_{r,j}|},\ \mathrm{CFL}\, \frac{\Delta x_r}{\gamma_r |1 - 4 f^k_r|} \right\}. \tag{63}$$

In all numerical simulations we use $\mathrm{CFL} = 1/2$.

3.7 Numerical comparison of first-order and second-order methods

We compare the Rusanov first-order finite volume scheme Section 3.3 with the second-order extension from Section 3.5 on a single road, using the scaled LWR model (9), with the Greenshields flux (8) and characteristic speed (11). In all experiments we set $\gamma = 1$ and use the same CFL time-stepping rule (63). The first-order method uses the Rusanov flux (49), while the second-order method combines minmod-limited piecewise linear reconstruction Section 3.5.1 with SSPRK2 time integration Section 3.5.2. We first consider Riemann problems with piecewise constant initial data, illustrating the behaviour of the two schemes in the presence of shocks and rarefaction waves. We then study a smooth test problem for which the solution remains smooth over the time interval of interest, allowing us to examine accuracy and convergence properties in the absence of discontinuities.

3.7.1 Finite volume solutions of Riemann problems

The first test consists of two Riemann-type initial conditions on $x \in [0, 1]$ with a single jump at $x_0 = 1/2$, evolved to $t = 0.5$. We fix $N = 20$ cells and use constant inflow at $x = 0$ consistent with the left state via the boundary flux $f(\rho_L)$, while the outflow boundary is handled by the free outflow rule as described in Section 2.5. The initial data are

$$\rho_0(x) = \begin{cases} \rho_L, & x < x_0, \\ \rho_R, & x > x_0, \end{cases} \qquad x_0 = \tfrac{1}{2}, \tag{64}$$

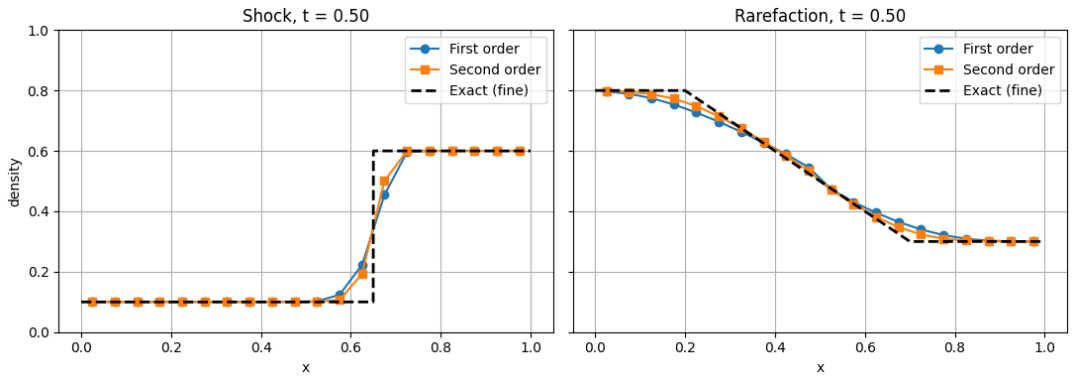

Figure 9: Riemann experiments at $t = 0.5$. Solutions of (64) for a shock with $(\rho_L, \rho_R) = (0.10,\ 0.60)$ (left) and a rarefaction with $(\rho_L, \rho_R) = (0.80,\ 0.30)$ (right). We compare a first-order Rusanov scheme (49) and a second-order finite volume scheme (see Section 3.5) with minmod limiter (54), against the exact entropy solution.

with the parameter choices $(\rho_L, \rho_R) = (0.10,\ 0.60)$ in the shock case, and $(\rho_L, \rho_R) = (0.80,\ 0.30)$ in the rarefaction case. The corresponding exact entropy solutions are the standard Riemann solutions from Section 2.4: the shock speed is given by the Rankine–Hugoniot relation (18), and the rarefaction fan is determined by the characteristic speeds $\gamma f'(\rho_L)$ and $\gamma f'(\rho_R)$ from (11). The results are shown in Figure 9, where we plot both numerical solutions against the exact entropy solution. Both schemes reproduce the correct qualitative structure of the solution. The second-order method provides a sharper approximation of both the shock front and the rarefaction profile, while remaining stable and free of spurious oscillations.

3.7.2 Convergence comparison for a smooth solution

To assess the convergence behaviour of the finite volume schemes in the absence of discontinuities, we consider a smooth test problem for which the solution of (9) remains smooth over the time interval of interest. The goal of this experiment is to study the convergence rate under grid refinement when no shock formation occurs. The initial condition is chosen as a smooth density profile,

$$\rho(x, 0) = 0.5 + 0.1 \exp\left(-\frac{(x - 0.5)^2}{\sigma^2}\right), \qquad \sigma = 0.15, \tag{65}$$

which represents a small Gaussian perturbation around the constant state $\rho = 0.5$. The amplitude is chosen sufficiently small to ensure that characteristics do not intersect before the final time. Boundary conditions are imposed by fixing the ghost-cell values to $\rho = 0.5$ at $x = 0$ and $x = 1$.

The solution is evaluated at a fixed time $t = 0.30$, which lies strictly before the shock formation time. In this regime, the exact solution can be computed using the method of characteristics, (12).

Numerical solutions are computed on a sequence of uniform grids with mesh size $\Delta x = 1/N$, using both the first-order finite volume scheme and the second-order scheme with minmod limiter. A visual comparison of the numerical solutions for selected grid resolutions is shown in Figure 11, where the right panel provides a zoom into the upper part of the solution to highlight small differences between the methods.

To quantify the convergence behaviour, we compute the discrete L^1 error at the final time t_M,

$$E(\Delta x) = \sum_j \left|\rho_j^M - \rho(x_j, t_M)\right| \Delta x,$$

where ρ_j^k denotes the numerical cell average in cell C_j and x_j is the corresponding cell centre. The observed convergence rate is estimated from successive refinements according to

$$p \approx \frac{\log\big(E(\Delta x)/E(\Delta x/2)\big)}{\log 2}.$$

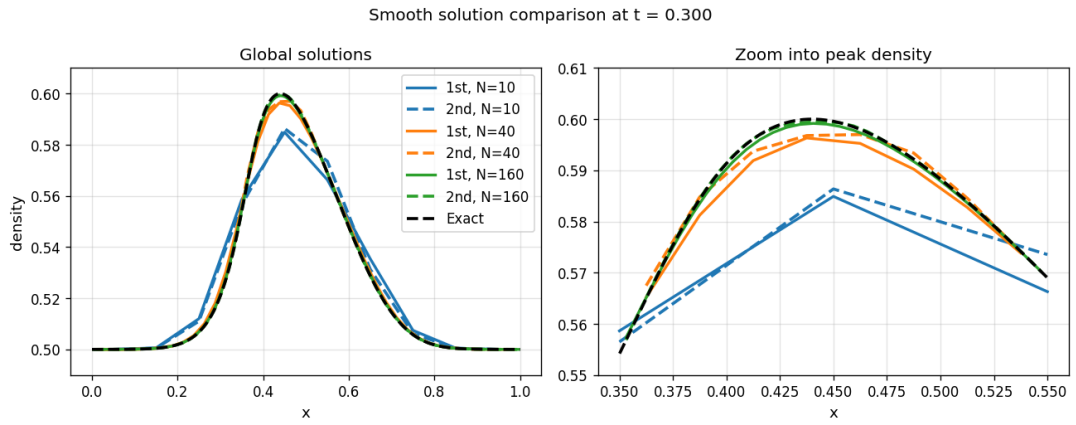

Figure 10: Smooth-solution experiment at a fixed time. Solutions of (65) computed using a first-order Rusanov scheme (49) and a second-order finite volume scheme (see Section 3.5) with minmod limiter (54) for several grid resolutions.

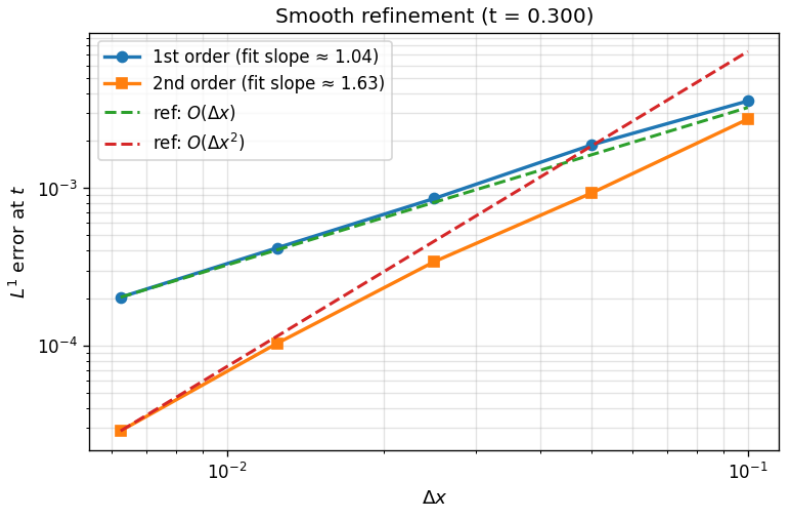

Figure 11: Convergence study for the smooth test problem. L^1 error as a function of grid spacing for first-order and second-order finite volume schemes.

The resulting error curves and estimated convergence rates for both schemes are presented in Figure 11. The first-order scheme gives an approximately linear decay of the error under grid refinement, with an observed convergence rate close to one. This is consistent with the theoretical first-order accuracy of the method.

For the second-order scheme, the error decreases significantly faster as the grid is refined. The estimated convergence rates are consistently larger than one and approach values between 1.6 and 1.8 on the finer grids. Although the asymptotic second-order rate is not fully attained, the results clearly confirm an improved convergence behaviour compared to the first-order scheme.

The slight reduction from the ideal second-order rate can be attributed to the use of slope limiting and the finite resolution of the grids. Overall, the experiment demonstrates that the second-order finite volume scheme achieves higher accuracy in smooth regimes, in agreement with the theoretical expectations.

4 Neural networks as junction models

For the road–network systems considered in Section 2.6, the traffic dynamics on each road are fully determined by the LWR equation together with the finite volume discretization. Once these components are fixed, the only modeling freedom lies in the junction model K, which specifies how fluxes are exchanged between incoming and outgoing roads. Since analytical junction models rely on simplifying assumptions and may not accurately reflect observed driving patterns, the junction constitutes a suitable candidate for data–driven modeling.

In this section, the junction model is represented by a neural network. The neural network defines a mapping from the boundary states at the junction to the corresponding junction fluxes and replaces the analytical junction coupling introduced in Section 2.5. No changes are made to the LWR model on the roads or to the numerical discretization. The resulting solver therefore remains a standard conservation–law–based finite volume method, with learning confined to the junction coupling.

The remainder of this section introduces the neural network architecture and describes its integration into the numerical solver. Training scenarios, loss functions, and differentiability properties of the resulting hybrid solver are then discussed, forming the basis for the numerical experiments in Section 5.

4.1 Neural networks

We define the class of neural networks that will be used later as parametric models for unknown junction flux functions. A neural network is a function that maps a vector of inputs to a vector of outputs, $\Phi_\theta : \mathbb{R}^n \to \mathbb{R}^m$, and depends on a finite set of real parameters. These parameters are adjusted during training.

A neural network is built from a sequence of *layers*. Each layer consists of an affine linear map followed by a componentwise nonlinear function called an *activation function*. Let $L \in \mathbb{N}$ be the total number of layers. For each layer $\ell = 1, \dots, L$, we introduce a weight matrix $W_\ell \in \mathbb{R}^{d_{\ell+1} \times d_\ell}$ and a bias vector $b_\ell \in \mathbb{R}^{d_{\ell+1}}$. The corresponding affine linear map is

$$A_\ell(x) = W_\ell x + b_\ell.$$

The integers $d_1 = n$ and $d_{L+1} = m$ are the input and output dimensions of the network. The layers $\ell = 2, \dots, L-1$ are referred to as the *hidden* layers, and their widths are given by $d_2, \dots, d_L$.

Each affine map is followed by an activation function $\sigma_\ell : \mathbb{R} \to \mathbb{R}$, which is applied componentwise to vectors in $\mathbb{R}^{d_{\ell+1}}$. The role of the activation functions is to introduce nonlinearity into the network. Without them, the network would be a composition of affine linear maps only, and therefore would reduce to a single affine linear mapping. In that case, no nonlinear input–output relations could be represented.

The activation functions may differ from layer to layer. They are continuous mappings that introduce nonlinearity into the network and are typically smooth almost everywhere. A common nonsmooth choice is the *rectified linear unit* (ReLU)

$$\sigma_{\mathrm{ReLU}}(x) = \max\{0, x\}, \tag{66}$$

A smooth alternative to ReLU that is often used when differentiability is required is the *Softplus* function

$$\sigma_{\mathrm{sp}}(x) = \log\bigl(1 + e^x\bigr). \tag{67}$$

With this notation, the full neural network is the composition

$$\Phi_\theta(x) = \sigma_L \circ A_L \circ \sigma_{L-1} \circ A_{L-1} \circ \cdots \circ \sigma_1 \circ A_1(x), \tag{68}$$

where $\theta = \{W_\ell, b_\ell\}_{\ell=1}^{L}$ denotes the collection of all weights and biases. The vector θ is called the parameter vector of the network. The layer widths $\{d_\ell\}$ and the activation functions $\{\sigma_\ell\}$ are fixed

in advance and define the network architecture. Since each affine map A_ℓ is defined by a full weight matrix $W_\ell \in \mathbb{R}^{d_{\ell+1} \times d_\ell}$, every component of one layer is connected to every component of the next. Moreover, the network consists of a finite sequence of layers without feedback connections. Neural networks of this form are referred to as *fully connected feedforward neural networks*.

For any fixed parameter vector θ, the mapping Φ_θ is continuous and differentiable almost everywhere with respect to both the input x and the parameters θ. This follows from the fact that Φ_θ is a finite composition of affine maps and activation functions with the same regularity. This property will be fundamental later, since it allows us to apply gradient-based optimization to loss functions that depend on the output of a numerical solver.

Neural networks are commonly used as parametric models for unknown functions. Given a function $f : \mathbb{R}^n \to \mathbb{R}^m$ defined on a compact domain $K \subset \mathbb{R}^n$, one chooses a network architecture and seeks parameters θ such that $\Phi_\theta(x)$ provides a good approximation of $f(x)$ for $x \in K$.

The theoretical justification for this practice is provided by the *Universal Approximation Theorem* (UAT). One version of the theorem states that if the activation function is continuous and non-polynomial on any interval, then the set of feedforward neural networks with at least one hidden layer is dense in $C(K)$ for any compact set $K \subset \mathbb{R}^d$ [21]. That is, for every continuous function $f : K \to \mathbb{R}^m$ and every $\varepsilon > 0$, there exists a parameter vector θ such that

$$\sup_{x \in K} \|\Phi_\theta(x) - f(x)\| < \varepsilon.$$

4.2 Hybrid solver and reference solver

We consider a road network system with a single junction, as defined in Section 2.6. We assume that the true traffic dynamics are governed by this system, consisting of the LWR model on each road coupled through a physical junction model K.

Since real traffic measurements are not available, all data used in this work are generated synthetically. This setting allows us to focus on approximation and optimization effects, without the additional complications of measurement noise or uncertainty. We define a *reference solver* by discretizing the road network system using a finite volume method. Specifically, the LWR equations on each road are discretized using a standard finite volume scheme, such as the Godunov flux (Section 3.2), the Rusanov flux (Section 3.3), or a second–order extension (Section 3.5), combined with the time stepping rule and boundary condition treatment described in Section 3.6. The same finite volume scheme is applied on all roads. At the junction, the numerical fluxes are computed using the physical junction model K. The resulting reference solver is used to generate all observations.

The *hybrid solver* is defined by modifying only the junction coupling in the reference solver. The spatial discretization, grid, time stepping, numerical fluxes, and external boundary conditions are kept identical. The physical junction model K is replaced by a neural network approximation

$$K_\theta \colon [0,1]^{n+m} \to \mathbb{R}^{n+m},$$

where θ denotes the trainable parameters. The map K_θ is represented by a neural network as defined in (68) and serves as a data–driven surrogate for the analytical junction model. We reffer to K_θ, as the *neural junction model*.

In contrast to the junction model K, the neural junction model K_θ does not, in general, enforce positivity, mass conservation, or capacity constraints. Learning is therefore confined to the junction coupling, while the conservation law structure and shock–capturing properties of the finite volume solver on each road are preserved.

Since K is continuous on the compact set $[0,1]^{n+m}$, the UAE, see Section 4.1, guarantees that, for a sufficiently expressive network architecture, K_θ can approximate K arbitrarily well on this domain.

4.3 Differentiability of the hybrid solver

The hybrid solver defined in Section 4.2 differs from the reference finite volume network solver only in the junction coupling. The physical junction model K is replaced by a neural network K_θ, while all other components of the solver, including the grid, time stepping, numerical fluxes, and external boundary conditions, are unchanged.

For finite volume network solvers with demand–supply junction rules, it is well established that the discrete update can be written as a composition of elementary operations and therefore admits *automatic differentiation*, see for example [11]. Such solvers are differentiable almost everywhere in the sense required for automatic differentiation, with possible non-differentiability occurring only at switching points of min and max operations.

In the hybrid solver, the junction fluxes are computed by the neural network K_θ. For standard network architectures, K_θ is differentiable almost everywhere with respect to its parameters θ. Replacing K by K_θ therefore preserves the almost-everywhere differentiability of the overall solver.

The use of ghost cells to impose junction and external boundary fluxes does not affect this argument. Ghost cells are constructed so that the numerical flux at the boundary matches a prescribed value, while the same numerical flux function is used throughout the domain. Junction and boundary updates therefore have the same regularity properties as interior finite volume updates.

It follows that the hybrid solver defines a mapping from the neural network parameters θ to the discrete solution that is differentiable almost everywhere. The loss functions introduced later in Section 5, which are based on squares of fluxes or densities, are compositions of this mapping with algebraic operations and hence inherit the same almost-everywhere differentiability with respect to θ.

4.4 Training scenarios and loss functions

Let $\{\rho_{r,j}^k\}$ denote the reference solution produced by a reference solver, described in Section 4.2, while $\{\hat{\rho}_{r,j}^k\}$ denotes the corresponding hybrid solution obtained by replacing the junction model K with the neural junction model K_θ. All quantities produced by the hybrid solver are marked by a hat. Which components of the reference solution are assumed to be available for training depends on the specific scenario. We call the avaliable data "observations".

We consider three different observation settings. In the first scenario, both the junction boundary densities and the corresponding junction fluxes are available. In the second scenario, the full discrete traffic $\{\rho_{r,j}^k\}$ state is available in space and time, but the junction fluxes are not directly available. In the third scenario, only sparse state observations are available at a finite number of road cells and discrete time instances.

In all scenarios considered below, the observed data generate a dataset $\mathcal{D}$. Each element $d \in \mathcal{D}$ corresponds to one data row and contains the observed quantities together with the information needed to compute the associated hybrid-solver value.

Given a data row $d \in \mathcal{D}$, the hybrid solver computes the corresponding value using the neural junction model K_θ. The *local loss* associated with d is defined as

$$\ell_d(\theta) = \| \text{ observed value} - \text{computed value } \|_2^2,$$

which is the squared error between the reference solution and the hybrid solver. Such squared-error local losses are standard in data-driven approximation and learning problems. The overall performance of K_θ is measured by the *mean square error* (MSE) over the entire dataset,

$$\mathcal{L}(\theta) = \frac{1}{|\mathcal{D}|} \sum_{d \in \mathcal{D}} \ell_d(\theta).$$

Training the neural network consists of finding parameters θ that minimize this loss,

$$\theta_{\min} = \arg\min_\theta \mathcal{L}(\theta). \tag{69}$$

In learning problems of the form (69), it is common to introduce *regularization*, meaning additional assumptions that bias the learning process toward physically meaningful solutions. Regularization does not necessarily turn the problem into a constrained optimization problem, but is often implemented by augmenting the loss function to penalize violations of desired properties. For instance, one may introduce regularization terms that encourage the junction fluxes to satisfy a mass balance condition, as described in (23). Another possible approach is to restrict the image of the neural network through the choice of activation functions in the output layer, such as using ReLU (66) or softplus (67) activations to enforce nonnegativity of fluxes. Since this alters the set of realizable outputs, it may also be interpreted as a form of architectural regularization.

In the context of data–driven junction models, regularization has been used to ensure consistency with macroscopic traffic flow theory. For instance, in [22], the neural network is combined with a problem–specific parametrization of the admissible junction flux set, thereby enforcing mass conservation and demand–supply constraints by construction.

In this work, no explicit regularization is imposed. The neural junction model K_θ is trained solely by minimizing the data–misfit loss (69), without additional penalty terms or hard constraints. This choice allows us to study how a purely data–driven junction model behaves when embedded into an otherwise classical finite volume traffic network solver.

4.5 Training procedure

In Sections 5.1 to 5.3, we introduced three training scenarios and their associated loss functions, defined in (73), (76) and (77). Each of these loss functions measures the discrepancy between the hybrid and reference solvers under different observation settings. Recall that in all cases, training the neural junction model K_θ corresponds to solving the optimization problem (69), where $\mathcal{L}(\theta)$ denotes the chosen loss function.

The optimization problem (69) is high-dimensional and *nonconvex*. In practice, one does not attempt to compute an exact minimizer, but instead applies iterative optimization methods to obtain a sufficiently good approximation of a minimizer of $\mathcal{L}(\theta)$. Iterative optimization methods may broadly be divided into gradient-free and gradient-based approaches. Gradient-free methods do not require derivative information, but are generally inefficient for high-dimensional problems. Gradient-based methods exploit derivative information to guide the optimization and are therefore the standard choice for training neural networks.

The use of gradient-based optimization requires the loss function to be differentiable with respect to the parameters θ. In Section 4.3, we argued that the hybrid solver is differentiable almost everywhere with respect to θ. This property carries over to the loss functions considered here. In the first training scenario, the loss depends directly on the output of the neural junction model K_θ through a squared error term. In the second and third scenarios, the loss depends on K_θ through one finite volume update or through an entire network simulation, respectively, followed by sums of squared differences with respect to fixed reference values. Since these operations preserve differentiability almost everywhere, the loss functions $\mathcal{L}(\theta)$ are differentiable almost everywhere with respect to θ.

Having established that gradient-based optimization methods are applicable to all three training scenarios, what remains is to select a suitable optimization algorithm. The most basic gradient-based method is *gradient descent* (GD), which updates the parameters θ iteratively according to

$$\theta^{(n+1)} = \theta^{(n)} - \alpha^{(n)} \nabla_\theta \mathcal{L}(\theta^{(n)}),$$

where $\alpha^{(n)} > 0$ denotes the learning rate. That is, the update follows the direction of steepest descent of $\mathcal{L}(\theta)$ at $\theta^{(n)}$.

Evaluating the loss $\mathcal{L}(\theta)$ and its gradient at every iteration may be computationally expensive, especially when the loss is defined through a finite volume simulation. A common alternative is therefore *stochastic gradient descent* (SGD), where the full loss $\mathcal{L}(\theta)$ is replaced by a local loss $\ell_d(\theta)$ associated with a single data point $d \in \mathcal{D}$. More generally, one may use *mini-batch* SGD,

where the update is computed from an average of local losses over a subset of data points. Here, an iteration refers to a single parameter update based on one mini-batch, while an *epoch* denotes one full pass through the training dataset. These stochastic variants reduce the cost per update and introduce additional stochasticity into the optimization.

In this work, we use the *Adaptive Moment Estimation* (Adam) optimizer [23], a gradient-based optimization method with adaptive step sizes. The Adam parameter update is given by

$$\theta^{(n+1)} = \theta^{(n)} - \alpha \frac{\hat{m}_n}{\sqrt{\hat{v}_n} + \varepsilon}, \tag{70}$$

where $\alpha > 0$ denotes a global step length, also known as the learning rate, and $\varepsilon > 0$ is a small constant added for numerical stability. Here, $\hat{m}_n$ and $\hat{v}_n$ are bias-corrected estimates of the first and second moments of the gradient, defined by

$$m_n = \beta_1 m_{n-1} + (1-\beta_1)\,\nabla_\theta \mathcal{L}(\theta^{(n)}), \qquad v_n = \beta_2 v_{n-1} + (1-\beta_2)\,\nabla_\theta \mathcal{L}(\theta^{(n)})^2,$$

$$\hat{m}_n = \frac{m_n}{1-\beta_1^n}, \qquad \hat{v}_n = \frac{v_n}{1-\beta_2^n},$$

with decay parameters $\beta_1, \beta_2 \in (0,1)$, where the square is taken componentwise. In other words, Adam combines the advantages of momentum-based methods and adaptive step-size strategies. The first moment estimate m_n acts as a momentum term, smoothing the descent direction by averaging gradients over previous iterations, while the second moment estimate v_n rescales the update to account for the variability of the gradient components. As a result, Adam automatically adjusts the effective step length for each parameter, leading to stable and efficient training in high-dimensional and nonconvex optimization problems.

Finally, we comment on how the gradients required by the optimization algorithms are obtained in practice. The hybrid solver defines the loss function $\mathcal{L}(\theta)$ as a finite composition of elementary operations, including finite volume updates, junction coupling through the neural junction model K_θ, and evaluations of squared-error loss terms. Since each of these operations is differentiable almost everywhere, the overall loss function is differentiable almost everywhere with respect to the parameters θ. As a consequence, the gradient $\nabla_\theta \mathcal{L}(\theta)$ can be computed by repeated application of the chain rule. A general discussion of automatic differentiation in this setting can be found in [24].

In practice, gradients are evaluated using *backpropagation* through reverse-mode automatic differentiation. During the forward pass, the hybrid solver records the sequence of elementary operations performed in the simulation. In the backward pass, derivatives are propagated in reverse order along this evaluation trace to compute $\nabla_\theta \mathcal{L}(\theta)$. All training experiments in this report are implemented using `PyTorch`, which provides built-in support for reverse-mode automatic differentiation and gradient-based optimization [25].

The computational cost of training depends strongly on the chosen loss function. For the direct junction loss introduced in Section 5.1, gradients are computed only through the neural junction model K_θ, which makes each training step computationally inexpensive. In the second training scenario, gradients propagate through a single finite volume update, leading to a moderate increase in computational cost. In contrast, the simulation-based loss defined in Section 5.3 requires backpropagation through the entire hybrid network simulation over the full time horizon. This significantly increases both computational time and memory usage, since all intermediate states must be stored during the forward pass in order to enable automatic differentiation. Similar computational and memory challenges are well known when training neural networks embedded in numerical solvers for PDE-based models, where gradients must be propagated through time-dependent simulations [26]. In that work, which focuses on PINNs, the authors report that substantially lower loss values can be achieved using alternative optimization algorithms such as *L-BFGS* and second-order methods in cases where Adam stagnates.

5 Numerical experiments

This section presents numerical experiments illustrating the behavior of the hybrid road–network solver introduced in Section 4.2. The experiments are organized according to three observation settings of increasing difficulty, reflecting different levels of information available for training the junction model.

We begin with a fully supervised setting in which both the junction densities and the corresponding junction fluxes are assumed to be known. In this case, the neural junction model can be trained directly on input–output data at the junction. This idealized scenario serves as a baseline and isolates the approximation capabilities of the neural network from the surrounding numerical solver.

We then consider a setting in which the full discrete traffic state is observed in space and time, but the junction fluxes are not directly available. The neural junction model is instead trained indirectly through its influence on the finite volume updates at the junction.

Finally, we study a sparse observation setting in which measurements are available only at a limited number of spatial locations and time instances. This setup reflects a more realistic data scenario in road–network applications. In this case, the junction model is trained solely through its effect on the global network dynamics, making this the most challenging setting considered.

All numerical experiments presented in this section and Section 3.7 are based on a simulation framework implemented in Python for this report. The finite volume solvers, junction handling, and hybrid coupling between the numerical model and the neural junction model are implemented explicitly by the author, without relying on established numerical solver libraries. The implementation is inspired by, reuses and builds upon code and ideas from [11].

5.1 Training with a known junction model

In this first scenario, the physical junction model K is assumed to be known. All training data are generated using the reference finite volume road–network solver described in Section 4.2. This solver models the traffic dynamics on each road segment and enforces coupling conditions at junctions through the model K.

At each discrete time level $k \in \mathcal{T} = \{0, \dots, M\}$, the solver provides the discrete density values at the junction boundaries. These values are collected in the junction boundary density vector

$$\boldsymbol{\rho}_K^k = \big((\rho_{r,b_r N}^k)_{r\in\mathcal{I}}, (\rho_{r,0}^k)_{r\in\mathcal{O}}\big),$$

as introduced in Section 3.6. Here, $\mathcal{I}$ and $\mathcal{O}$ denote the sets of incoming and outgoing roads, respectively. The components $\rho_{r,b_r N}^k$ and $\rho_{r,0}^k$ represent the densities at the grid cells adjacent to the junction on each incoming and outgoing road. Consequently, the vector $\boldsymbol{\rho}_K^k$ contains all density information required by the junction model at time step k.

Given these boundary densities, the physical junction model K computes the corresponding junction fluxes, which describe how traffic flows through the junction. Using the finite volume solver, we therefore obtain pairs consisting of boundary densities and their associated fluxes at every time step. The resulting training dataset is defined as

$$\mathcal{D}_{\text{flux}} = \{(\boldsymbol{\rho}_K^k, K(\boldsymbol{\rho}_K^k))\}_{k=1}^M \subset [0,1]^{n+m} \times \mathbb{R}_{\geq 0}^{n+m}. \tag{71}$$

Each element of $\mathcal{D}_{\text{flux}}$ corresponds to one observation of the junction behavior. The first component, $\boldsymbol{\rho}_K^k$, lies in the normalized density space $[0,1]^{n+m}$, while the second component, $K(\boldsymbol{\rho}_K^k)$, contains the nonnegative fluxes associated with the same junction configuration.

From a machine learning perspective, this dataset naturally defines a supervised learning problem. In supervised learning, one is given input–output pairs and seeks to learn a model that reproduces the underlying mapping between them. In the present setting, the inputs are the observed junction densities $\boldsymbol{\rho}_K^k$, and the outputs are the corresponding junction fluxes computed by the physical model K.

The neural junction model K_θ is trained on $\mathcal{D}_{\text{flux}}$ with the objective of approximating this input–output relation. In other words, K_θ is intended to serve as a data-driven surrogate for the known physical junction model K. The local loss at time level k is defined as

$$\ell_k(\theta) = \left\| K(\boldsymbol{\rho}_K^k) - \hat{K}_\theta(\boldsymbol{\rho}_K^k) \right\|_2^2. \tag{72}$$

It measures the discrepancy between the fluxes produced by the physical junction model K and those predicted by the neural junction model $\hat{K}_\theta$ for the same junction boundary densities. Recall that the hat is to specify that the value is produced by the hybrid solver, or in this instance directly from the neural junction model. Here and in the following, the hat notation indicates quantities produced by the hybrid solver. In particular, $\hat{K}_\theta(\cdot)$ denotes the fluxes predicted by the neural junction model.

The corresponding MSE over the entire dataset is given by

$$\mathcal{L}_{\text{flux}}(\theta) = \frac{1}{M+1} \sum_{k=0}^{M} \left\| K(\boldsymbol{\rho}_K^k) - \hat{K}_\theta(\boldsymbol{\rho}_K^k) \right\|_2^2, \tag{73}$$

which averages the local loss over all available time levels.

We emphasize that, in this first scenario, it is not necessary to generate the dataset $\mathcal{D}_{\text{flux}}$ by numerical simulation. Since the junction model K is assumed to be known or directly observable, the training data may also consist of measured input–output pairs $(\boldsymbol{\rho}_K, K(\boldsymbol{\rho}_K))$. Thus, the supervised learning setting requires knowledge of the junction mapping itself, but not of the full network dynamics.

If a finite volume solver is used to generate the data, it acts as a controlled data generator in which the physical model is known and all components of the dynamics except the junction model are fixed. This makes it possible to study the behavior of the neural junction model $\hat{K}_\theta$ in isolation.

For simplicity of notation, the dataset $\mathcal{D}_{\text{flux}}$ has been defined as if it were generated from a single simulation with fixed initial and boundary conditions. In practice, training can be performed on data obtained from multiple simulations with different initial and boundary conditions. In that case, the total dataset is given by

$$\mathcal{D}_{\text{flux}} = \bigcup_{s=1}^{S} \mathcal{D}_s,$$

where S denotes the number of simulations. The single-simulation description is used solely for ease of presentation.

5.1.1 Fitting directly on junction model

As discussed in Section 5.1, in principle the neural junction model could be trained on a large and diverse collection of input–output samples, generated for many different traffic configurations and covering a wide range of junction states. Such a setup would resemble a classical offline regression problem, in which the goal is to approximate the junction mapping independently of the surrounding network dynamics. However, this setting is only weakly related to practical traffic applications, where data are typically available as time series generated by a limited set of network configurations, and where access to complete junction observations is limited.

We therefore begin with an idealized supervised setting, not as a realistic training scenario, but as a controlled baseline that allows us to isolate and assess the approximation capabilities of the neural junction model itself. This setting provides a useful reference point before moving to more realistic observation regimes in which junction fluxes are no longer directly available.

We consider a single 2-to-2 junction with two incoming and two outgoing roads, and assume that traffic flow is governed by the system defined in Section 2.6. The junction is equipped with the physical junction model K defined in (30), using the turning fraction matrix

$$A = \begin{bmatrix} 0.25 & 0.75 \\ 0.60 & 0.40 \end{bmatrix}.$$

Road	Role	bL [m]	$\rho_{\max}$	$v_{\max}$ [km/h]	$\rho_0(x)$	$\rho_i^{\mathrm{ext}}(t)$
1	in	200	1	60	$\begin{cases} \frac{1}{3}, & x \le 100, \\ \frac{1}{2}, & x > 100 \end{cases}$	0
2	in	200	1	60	0.2	0.1
3	out	200	1	100	0.8	–
4	out	200	1	60	0	–

Table 1: Road-specific parameters for the training configuration of the 2-to-2 junction. Inflow fluxes use the same $v_{\max}$ and $\rho_{\max}$ as the road they enter.

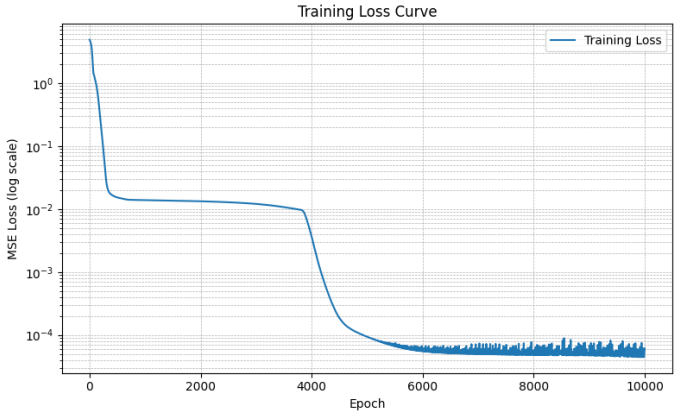

Figure 12: Supervised training loss for the neural junction model K_θ of the physical junction model K.

The corresponding speed limits, road lengths, maximal densities, initial density profiles, and external boundary inflow fluxes defining the reference configuration are summarized in Table 1.

To generate observation data, we use the reference solver defined in Section 4.2. Each road, of total length $bL = 200\,\mathrm{m}$, is constructed using $b = 4$ segments of length $L = 50\,\mathrm{m}$. Each segment is discretized into $N = 10$ finite volume cells, resulting in 40 cells per road. The simulation is performed over a time horizon of $T = 60\,\mathrm{s}$.

The reference solver uses the first-order finite volume scheme (49). From the resulting simulation, we sample the boundary density vectors $\boldsymbol{\rho}_K(t_k)$ at discrete times t_k and evaluate the corresponding junction fluxes $K(\boldsymbol{\rho}_K(t_k))$, yielding the dataset of junction observations $\mathcal{D}_{\mathrm{flux}}$, see (71).

The neural junction model K_θ is approximated by a fully connected feedforward neural network with four hidden layers, each of width 32. The input and output dimensions are both equal to 4, corresponding to the number of incoming and outgoing roads at the junction. All hidden layers use ReLU activation functions (66), while the output layer is linear.

The parameters θ are trained on $\mathcal{D}_{\mathrm{flux}}$ by minimizing the flux-based loss (73) using a full-batch Adam optimizer (70) with learning rate $\alpha = 10^{-3}$. Training is stopped after 10 000 epochs. Since a full-batch optimizer is used, one training epoch corresponds to a single optimization step of the Adam algorithm.

The supervised training loss shown in Figure 12 decreases during training and reaches values of order 10^{-5}. In the final phase of training, the loss fluctuates around a low level without any clear upward or downward trend. Overall, the loss curve in Figure 12 indicates that the training procedure converges successfully and that the neural junction model provides a consistent approximation of the observed junction fluxes under supervised training.

Road	Role	New initial density $\rho_0(x)$	New external inflow $\rho_i^{\text{ext}}(t)$
1	in	0.2	0
2	in	$\begin{cases} \frac{1}{3}, & x \le 100, \\ \frac{1}{2}, & x > 100 \end{cases}$	0.3
3	out	0	–
4	out	0.6	–

Table 2: Modified initial and external boundary conditions used for evaluation. All other network parameters are identical to the training configuration in Table 1. Inflow fluxes use the same $v_{\max}$ and $\rho_{\max}$ as the road they enter.

5.1.2 Hybrid solver performance and generalization

In practice, a learned junction model is not used in isolation, but as part of a full road–network simulation where its outputs directly influence the traffic dynamics on all connected roads. It is therefore important to assess not only how accurately the neural junction model K_θ fits observed junction fluxes, but also how it performs when embedded in the hybrid road–network solver.

In this experiment, the road network, junction configuration, numerical discretization, and training procedure are identical to those described in Section 5.1.1. To study how the learned neural junction model K_θ performs when used inside the full road–network hybrid solver, we carry out a checkpoint-based evaluation during training. Specifically, we save the neural network after $1{,}000, 2{,}000, \ldots, 10{,}000$ epochs. Each checkpoint is then inserted into the hybrid solver, also specified in Section 5.1.1, and a simulation is run on the same road network using the same numerical discretization as in the reference solver.

To compare the resulting network–level simulations between the reference solver and the hybrid solver, we evaluate discrepancies in the road–network states using the sparse loss introduced in (77). Observations are taken at times $\{t_k\}_{k\in\mathcal{T}_{\text{obs}}} = \{0, 1, \ldots, 60\}$, corresponding to one measurement per second throughout the simulation. The observation locations are placed in the boundary cell adjacent to the junction, mimicking toll-station density–measurements close to the junction.

To test performance under different conditions, we also repeat the same checkpoint–based evaluation using a second set of initial and boundary conditions, while keeping the road network and junction configuration unchanged. The modified initial conditions and external inflow boundary conditions are summarized in Table 2.

The training loss $\mathcal{L}_{\text{flux}}$ (73), together with the hybrid simulation errors measured by the sparse loss $\mathcal{L}_{\text{sparse}}$ (77), evaluated at each saved checkpoint of the hybrid solver, are shown in Figure 13. For the training initial and boundary conditions, we observe that as the discrepancy between the physical junction model K and its neural approximation K_θ decreases, the error of the full hybrid simulation also decreases. In this experiment, reductions in the junction-level flux loss therefore correlate with reductions in the network-level simulation error.

This behavior occurs roughly halfway through training, around epoch 5 000. Up to this point, both $\mathcal{L}_{\text{flux}}$ and $\mathcal{L}_{\text{sparse}}$ decrease steadily. Afterward, the supervised flux loss $\mathcal{L}_{\text{flux}}$ no longer decreases monotonically, but instead begins to oscillate. The peaks of these oscillations remain at approximately the same level as the value of $\mathcal{L}_{\text{flux}}$ at the epoch where the simulation error $\mathcal{L}_{\text{sparse}}$ stops decreasing.

When the same checkpoints are evaluated under modified initial and boundary conditions (Table 2), the hybrid simulation errors measured by the sparse loss $\mathcal{L}_{\text{sparse}}$ are substantially larger for all models. This shows that the supervised training captures the junction behavior for the specific initial and boundary conditions used during training, but does not generalize well to different conditions.

To illustrate the origin of this behavior, we compare the junction states encountered under the two sets of initial and boundary conditions using data generated by the reference solver. Let $\boldsymbol{\rho}_K(t) \in \mathbb{R}^4$ denote the vector of traffic densities on the roads adjacent to the junction at time t.

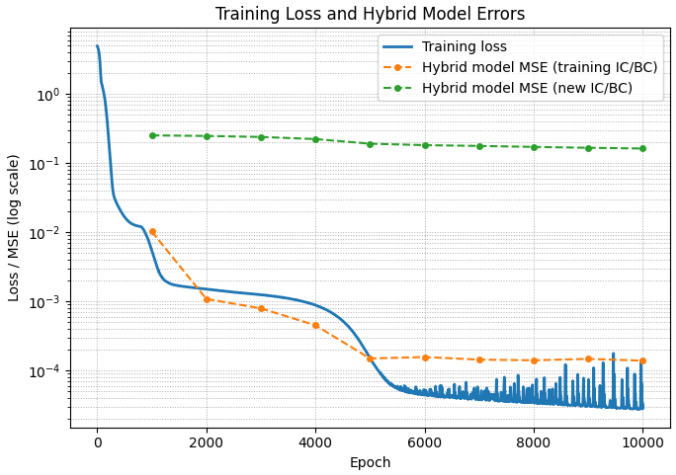

Figure 13: Supervised training loss and hybrid simulation errors evaluated on the training initial and boundary conditions and on a modified set of initial and boundary conditions. Each data point corresponds to a model checkpoint saved every 1 000 training epochs.

Since the supervised flux loss $\mathcal{L}_{\text{flux}}$ in (73) is evaluated only at junction states produced by a single training simulation, the training data cover only a limited range of such junction density vectors.

Figure 14 provides a two-dimensional visualization of the junction density vectors $\boldsymbol{\rho}_K(t)$. The four-dimensional data are projected onto the first two principal components using *principal component analysis* (PCA), which is a standard linear method for visualizing high-dimensional data [27]. Each point in the figure represents one junction state generated by the reference solver at a given time.

The junction states observed during training occupy only a limited region of the projected space. By contrast, the states generated under the modified initial and boundary conditions lie in a different region, with little overlap with the training states. This separation helps explain the poor performance of the hybrid solver under the modified conditions.

The neural junction model K_θ is trained only on junction states encountered during the training simulation and is therefore required to extrapolate when evaluated on states that differ significantly from those seen during training. This experiment shows that accurate supervised fitting of the junction model alone is not sufficient to guarantee reliable performance in network-level simulations. As expected, achieving robustness across different conditions requires training data that cover a wider range of relevant junction density configurations.

5.2 Training on full state observations

In this scenario, the observed data consist of the full discrete traffic state

$$\{\rho_{r,j}^k\}_{r\in\mathcal{R},\, j\in\mathcal{X}_r,\, k\in\mathcal{T}},$$

generated by reference solver, described in Section 4.2. Here, $\mathcal{R}$ denotes the set of roads, $\mathcal{X}_r = \{1,\dots,b_rN\}$ is the set of cell indices on road r, and $\mathcal{T} = \{0,1,\dots,M\}$ is the set of discrete time levels.

We assume that the external boundary conditions and all quantities required to advance the network dynamics are known. In particular, any external inflow or outflow fluxes are assumed to be observable and are included in the data. Under these assumptions, the learning problem can be treated as an isolated identification task for the junction model based on full state observations.

We recall from Section 3.6 that, in the finite volume discretization, each road r has exactly one cell adjacent to the junction. This cell corresponds to the downstream boundary cell for incoming

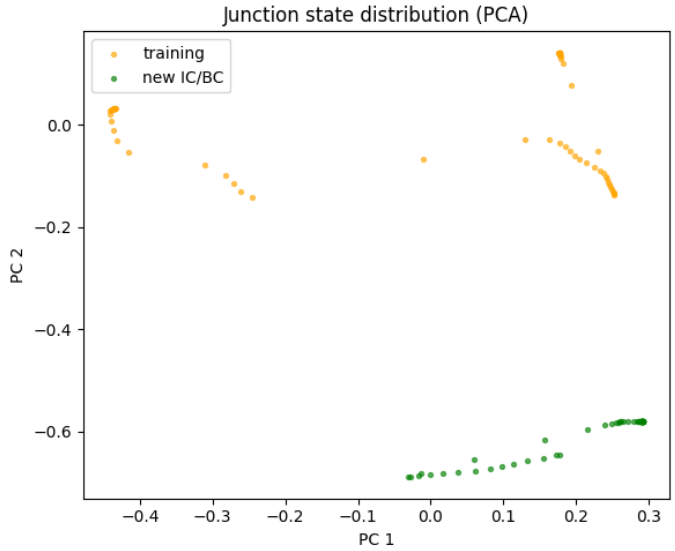

Figure 14: Projection of junction density states produced by the reference solver onto the first two principal components. Orange points correspond to junction states observed under the training initial and boundary conditions, while green points correspond to states arising under the modified initial and boundary conditions.

roads and the upstream boundary cell for outgoing roads, and is given by

$$j_r^J = \begin{cases} b_r N, & r \in \mathcal{I}, \\ 0, & r \in \mathcal{O}. \end{cases}$$

Recall also, that we collect all junction-adjacent cells in the set $\mathcal{K} = \{\, (r, j_r^J) \mid r \in \mathcal{R} \,\}$.

Throughout this subsubsection, we assume that the reference solver employs a first–order finite volume scheme. For such a scheme, the update of an interior cell on road r at position j from time level k to $k+1$ depends only on the three neighbouring cell averages at time k. We therefore introduce the corresponding *stencil*

$$s_{r,j}^k = (\rho_{r,j-1}^k,\ \rho_{r,j}^k,\ \rho_{r,j+1}^k),$$

which contains exactly the information needed to compute the update of $\rho_{r,j}^{k+1}$ for an interior cell.

For boundary cells, one of these values is replaced by the appropriate boundary flux condition described in Section 3. In particular, for the junction cell (r, j_r^J), one of the numerical fluxes in the update is replaced by the junction flux determined by the physical model K or its neural approximation K_θ.

For a first–order finite volume scheme, the complete stencil required to update cell (r, j) at time level k is given by

$$s_{r,j}^k = \begin{cases} (\rho_{r,j-1}^k,\ \rho_{r,j}^k,\ \rho_{r,j+1}^k), & \text{if } j \in \{1, \dots, b_r N - 1\}, \\ (\rho_{r,j_r^J-1}^k,\ \boldsymbol{\rho}_K^k), & \text{if } r \in \mathcal{I} \text{ and } j = j_r^J, \\ (\rho_{r,j_r^J+1}^k,\ \boldsymbol{\rho}_K^k), & \text{if } r \in \mathcal{O} \text{ and } j = j_r^J, \\ (\rho_{r,1}^k,\ f_r^{\text{in},k}), & \text{if } r \in \mathcal{I} \text{ and } j = 0, \\ (\rho_{r,b_r N-1}^k,\ f_r^{\text{out},k}), & \text{if } r \in \mathcal{O} \text{ and } j = b_r N. \end{cases} \tag{74}$$

The first case corresponds to interior cells, which use the standard three-point stencil. The second and third cases describe the junction-adjacent cells (r, j_r^J), whose update depends on the full

junction state $\boldsymbol{\rho}_K^k$ through the junction flux. The last two cases apply to external boundary cells, where one of the numerical fluxes is replaced by the prescribed external boundary fluxes $f_r^{\text{in},k}$ and $f_r^{\text{out},k}$, see (60) and (61).

In the first scenario (Section 5.1), the objective was to learn the junction fluxes directly. In the present setting, the goal is instead to learn the effect of the junction model through the resulting updates of the cell averages in the finite volume scheme. Since the full discrete traffic state $\{\rho_{r,j}^k\}$ is observed, together with the external boundary fluxes $\{f_r^{\text{in},k}, f_r^{\text{out},k}\}$, all quantities required to compute a single finite volume update are known.

In particular, for every cell (r, j), the complete stencil $s_{r,j}^k$ defined in (74) is fully observed. This makes it possible to evaluate both the reference update $\rho_{r,j}^{k+1}$, obtained using the physical junction model K, and the corresponding hybrid update $\hat{\rho}_{r,j}^{k+1}$, obtained by replacing K with its neural approximation K_θ, using the same state at time level k.

Each observed finite volume update from time level k to $k+1$, determined by the observations in the stencil $s_{r,j}^k$, defines one supervised training sample. Collecting all such updates over roads, cells, and time levels yields the dataset

$$\mathcal{D}_{\text{full}} = \big\{(r, j, k,\ s_{r,j}^k,\ \rho_{r,j}^{k+1}) \;\big|\; r \in \mathcal{R},\ j \in \mathcal{X}_r,\ k \in \mathcal{T} \setminus \{M\}\big\}.$$

Each element of $\mathcal{D}_{\text{full}}$ consists of the stencil $s_{r,j}^k$, together with the corresponding reference update $\rho_{r,j}^{k+1}$ produced by the finite volume solver using the physical junction model K.

For a given element of $\mathcal{D}_{\text{full}}$, we define the local loss

$$\ell_{r,j}^k(\theta) = \|\rho_{r,j}^{k+1} - \hat{\rho}_{r,j}^{k+1}\|^2, \tag{75}$$

which measures the discrepancy between the reference update $\rho_{r,j}^{k+1}$ and the corresponding update $\hat{\rho}_{r,j}^{k+1}$ obtained from the hybrid solver, in which the physical junction model is replaced by its neural approximation K_θ. This loss quantifies how accurately the learned junction model reproduces the effect of the physical junction model on the local evolution of the traffic state. Inserting the first–order finite volume updates for the reference solver and the hybrid solver gives

$$\begin{aligned}
\ell_{r,j}^k(\theta) &= \left\| \Big[\rho_{r,j}^k - \frac{\gamma_r\,\Delta t_k}{\Delta x_r}\big(F_{r,j+1/2}^k - F_{r,j-1/2}^k\big)\Big] - \Big[\rho_{r,j}^k - \frac{\gamma_r\,\Delta t_k}{\Delta x_r}\big(\hat{F}_{r,j+1/2}^k - \hat{F}_{r,j-1/2}^k\big)\Big] \right\|_2^2 \\
&= \left(\frac{\gamma_r\,\Delta t_k}{\Delta x_r}\right)^2 \left\| (F_{r,j+1/2}^k - \hat{F}_{r,j+1/2}^k) - (F_{r,j-1/2}^k - \hat{F}_{r,j-1/2}^k) \right\|_2^2 .
\end{aligned}$$

The numerical fluxes appearing in the update of cell (r, j) depend only on the variables contained in the stencil $s_{r,j}^k$ (74). For all interior cells and all external boundary cells, the relevant interfaces do not coincide with the junction, and the numerical fluxes of the reference and hybrid solvers therefore agree. Hence,

$$F_{r,j\pm1/2}^k = \hat{F}_{r,j\pm1/2}^k,$$

which implies that $\ell_{r,j}^k(\theta) = 0$ for all cells that are not adjacent to the junction.

For a junction-adjacent cell (r, j_r^J), exactly one of the two interfaces in the finite volume update lies at the junction. At this interface, the reference solver uses the physical junction flux determined by K, while the hybrid solver uses the corresponding flux predicted by the neural junction model $\hat{K}_\theta$. The second interface lies strictly inside the road segment and is therefore computed in the same way in both solvers from the same local densities contained in the stencil.

When comparing the reference and hybrid one-step updates for such a cell, the interior flux contribution is identical and cancels out. The difference between the two updates is therefore entirely due to the flux at the junction interface. As a result, the local loss for a junction-adjacent cell reduces to

$$\ell_{r,j_r^J}^k(\theta) = \left(\frac{\gamma_r\,\Delta t_k}{\Delta x_r}\right)^2 \Big|\big(K(\boldsymbol{\rho}_K^k)\big)_r - \big(\hat{K}_\theta(\boldsymbol{\rho}_K^k)\big)_r\Big|^2 .$$

This expression shows the key point of the full-state setting. Even though we train on one-step state updates, the only part of the update where the reference and hybrid solvers can differ is the junction interface. Consequently, the loss is nonzero only for junction-adjacent cells, and its magnitude is directly controlled by the mismatch between the physical junction fluxes and the fluxes predicted by $\hat{K}_\theta$.

It is important to emphasize that this does not mean that the training problem reduces to the same direct flux-matching setting considered in Section 5.1. In the present scenario, the local loss is still defined in terms of one-step state updates,

$$\ell_{r,j}^k(\theta) = \|\rho_{r,j}^{k+1} - \hat{\rho}_{r,j}^{k+1}\|^2,$$

and the hybrid update $\hat{\rho}_{r,j}^{k+1}$ must be computed by applying the finite volume scheme with the neural junction model $\hat{K}_\theta$. The dependence on θ therefore enters implicitly through the numerical update, and gradients are obtained by backpropagating through the finite volume update rule.

The derivation above shows that, for junction-adjacent cells, the magnitude of the local loss is directly linked to the discrepancy between the physical and neural junction fluxes. However, the learning problem remains fundamentally state-based, as supervision is provided through observed state transitions rather than direct flux observations.

Since $\ell_{r,j}^k(\theta) = 0$ for all cells not adjacent to the junction, the MSE loss is averaged only over the nonzero contributions, that is, over the cells in $\mathcal{K}$. The loss over the entire dataset is therefore

$$\begin{aligned}\mathcal{L}_{\text{full}}(\theta) &= \frac{1}{(|\mathcal{T}|-1)|\mathcal{K}|}\sum_{k=0}^{M-1}\sum_{(r,j_r^J)\in\mathcal{K}} \ell_{r,j_r^J}^k(\theta) \\ &= \frac{1}{M(n+m)}\sum_{k=0}^{M-1}\sum_{(r,j_r^J)\in\mathcal{K}} \left(\frac{\gamma_r\,\Delta t_k}{\Delta x_r}\right)^2 \left|(K(\boldsymbol{\rho}_K^k))_r - (\hat{K}_\theta(\boldsymbol{\rho}_K^k))_r\right|^2. \end{aligned} \tag{76}$$

We note that, for a first–order finite volume scheme, the full traffic state is not strictly required in order to learn the junction model. Since only the junction-adjacent cells depend on the junction flux, it would in principle suffice to observe only the densities in these cells. All other cell updates are unaffected by the choice of junction model and therefore contribute no error.

For higher–order schemes, such as the second–order method introduced in Section 3.5, the situation is less clear. The spatial stencil becomes wider, so additional cells may depend on the junction flux, and the intermediate RK stages introduce corrections that do not cancel as cleanly as in the first-order case. As a result, the reduction of the loss to a simple expression involving only the junction flux difference is no longer immediate. We do not analyse the higher-order case here.

5.2.1 Fitting on full dataset for a 2-to-3 junction

Compared with supervised flux fitting, this setting removes direct access to the junction model and instead trains it indirectly through its influence on the surrounding traffic dynamics. This reflects situations in which junction fluxes are not directly observable, but sufficiently rich state information is available from simulations or frequent measurements. At the same time, the full–state setting still provides strong supervision through the finite volume updates, and therefore serves as an intermediate step between idealized flux supervision and the sparse observation regime considered later.

We now consider a learning experiment based on the full-dataset fitting procedure introduced in Section 5.2, where no direct observations of junction fluxes are assumed to be available. Instead, the neural junction model is trained by embedding it into the numerical solver and minimizing the discrepancy between the resulting traffic state trajectories and those obtained from a fixed physical reference simulation.

We consider a single 2-to-3 junction with two incoming and three outgoing roads, and assume that traffic flow is governed by the system defined in Section 2.6. In contrast to the supervised

Road	Role	bL [m]	$\rho_{\max}$	$v_{\max}$ [km/h]	$\rho_0(x)$	$\rho_i^{\text{ext}}(t)$
1	in	400	1	30	$0.65\,\exp\big(-x/0.08\big)$	$0.2\,\exp\big(-((t-120)/40)^2\big)$
2	in	200	2	60	$\begin{cases} 0.7, & x \le 0.35, \\ 0.25, & x > 0.35 \end{cases}$	$\begin{cases} 0.05, & t < 150, \\ 0.2, & t \ge 150 \end{cases}$
3	out	400	1	50	0.4	–
4	out	400	1	20	0	–
5	out	200	1	40	0	–

Table 3: Road-specific parameters for the full-dataset training configuration of the 2-to-3 junction. Inflow fluxes use the same $v_{\max}$ and $\rho_{\max}$ as the road they enter.

flux-fitting experiment in Section 5.1.1, the junction model is not trained to match the physical fluxes directly, but instead to reproduce the time evolution of the traffic densities at the junction. The physical junction model K uses the fixed distribution matrix

$$A = \begin{bmatrix} 0.3 & 0.1 & 0.6 \\ 0.6 & 0.2 & 0.2 \end{bmatrix}.$$

The corresponding speed limits, road lengths, maximal densities, initial density profiles, and external boundary inflow fluxes defining the reference configuration are summarized in Table 3.

To generate reference data, we use a finite volume road–network solver as defined in Section 4.2. Each road is discretized using $N = 10$ cells per segment, resulting in 40 cells for roads with $b = 4$ segments and 20 cells for roads with $b = 2$ segments. The simulation is performed over a time horizon of $T = 300\,\text{s}$.

The reference solver employs a first–order Rusanov finite volume scheme, (49), together with the physical junction model K defined in (30). From this simulation, we extract the boundary density vectors $\boldsymbol{\rho}_K(t_k)$ at discrete times t_k, which are used to evaluate the discrepancy between the reference solver and hybrid solver during training.

The neural junction model K_θ is approximated by a fully connected feedforward neural network with four hidden layers, each of width 32. The input and output dimensions are both equal to 5, corresponding to the total number of incoming and outgoing roads at the junction. All hidden layers use ReLU activation functions (66), while the output layer is linear.

The parameters θ are trained by minimizing the density-based loss (76), which measures the mismatch between the density trajectories at junction-adjacent cells produced by the neural-network-controlled simulation and those obtained from the physical reference solution. Training is performed using a full-batch Adam optimizer with learning rate $\alpha = 10^{-2}$ over a fixed number of epochs.

Figure 15 shows the training loss associated with the full-dataset fitting procedure for the 2-to-3 junction. As in the supervised flux-fitting experiment of Section 5.1.1, the loss decreases steadily during training and stabilizes at a low level. At the level of the training objective, this indicates that the neural junction model is able to reproduce the reference one-step updates at the junction-adjacent cells when embedded in the full numerical model.

This behavior is consistent with the structure of the full-dataset loss defined in (76). As shown in its derivation, the loss is defined on one-step state updates, but its magnitude at junction-adjacent cells is directly linked to the discrepancy between the physical junction model K and its neural approximation $\hat{K}_\theta$, evaluated at the boundary density vectors $\boldsymbol{\rho}_K^k$. Importantly, this does not imply that the learning problem reduces to direct flux matching. The neural junction model is trained indirectly through the finite volume updates, and gradients are obtained by backpropagating through the numerical solver. Nevertheless, since successful approximation of the junction model was already demonstrated under direct flux supervision in Section 5.1.1, it is reasonable that a low training loss can also be achieved in this indirect setting for the training configuration considered here.

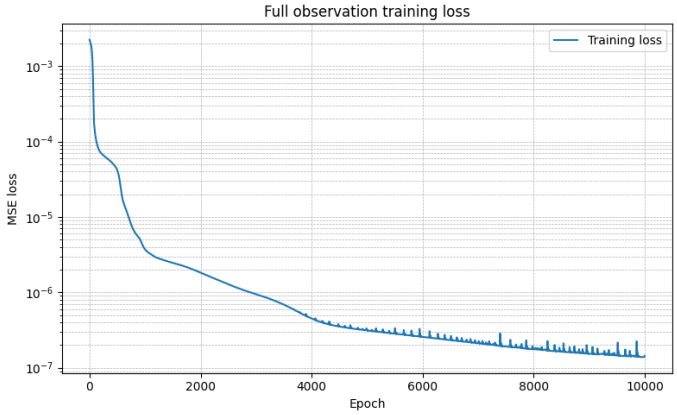

Figure 15: Full state training loss for the neural network approximation of the physical junction model K.

A notable difference between the present experiment and the supervised flux-fitting case, see Figure 12, is the magnitude of the training loss values. In the full-dataset setting, the loss reaches significantly smaller values. This behavior is primarily explained by the structure of the loss functions $\mathcal{L}_{\text{flux}}$ in (73) and $\mathcal{L}_{\text{full}}$ in (76). First, the error is averaged over all $n+m$ junction-adjacent cells at each time step, which reduces the overall magnitude of the loss compared with a loss defined directly on the complete junction flux vector. More importantly, each local error contribution is multiplied by the factor $(\gamma_r\,\Delta t_k/\Delta x_r)^2$, originating from the finite volume update at the junction. While the precise value of this factor depends on the numerical parameters, it is expected to significantly scale down the loss values. As a result, the absolute magnitude of the training loss in the full-dataset setting should not be interpreted as a direct indicator of improved junction model accuracy.

5.3 Training with sparse observations

In practice, the full traffic state on a road network is generally not available. Measurements are typically taken only at a few fixed locations, such as toll stations that record passing vehicles, and provide no information about the traffic density elsewhere on the road. As a result, the observations available for training cover only a small portion of the network.

To reflect this limitation, we assume that only a subset of the cell averages is accessible. In the fully observed setting of Section 5.2, the data consist of the complete discrete traffic state $\{\rho^k_{r,j}\}_{r\in\mathcal{R},\, j\in\mathcal{X}_r,\, k\in\mathcal{T}}$, whereas here we work only with the restricted set of observed values.

We denote the set of observed road–cell pairs by $\mathcal{C}_{\text{obs}} \subset \{(r,j) \mid r \in \mathcal{R},\, j \in \mathcal{X}_r\}$, and the set of observed time indices by $\mathcal{T}_{\text{obs}} \subset \mathcal{T}$. Thus the available training data consist only of the states. Thus the available training dataset is

$$\mathcal{D}_{sparse} = \{(r, j, k,\, \rho^k_{r,j}) \,|\, (r,j) \in \mathcal{C}_{\text{obs}},\ k \in \mathcal{T}_{\text{obs}}\}.$$

Throughout this setting, we assume that the initial conditions and the external boundary fluxes are known. This allows us to isolate the effect of the neural junction model on the network dynamics and to attribute discrepancies at the observed locations solely to the learned junction model. In principle, these quantities could also be treated as unknown and incorporated into the learning process, but we do not consider this extension here.

The key difficulty, compared with the full-state observation setting, is that the sparse data do not contain all quantities required by the finite volume scheme. For example, in a first–order scheme, the stencil in (74) shows that updating an interior cell (r,j) at time k requires the three values $(\rho^k_{r,j-1},\, \rho^k_{r,j},\, \rho^k_{r,j+1})$. However, these stencil values are generally not contained in $\mathcal{C}_{\text{obs}} \times \mathcal{T}_{\text{obs}}$ and must therefore be treated as unavailable.

As a consequence, the sparse observations do not provide sufficient information to compute the hybrid update $\hat{\rho}_{r,j}^{k+1}$ directly from the observed states at the previous time step.

To obtain the hybrid values $\hat{\rho}_{r,j}^{k}$ for all roads, cells, and time levels, we therefore simulate the entire network using the hybrid solver. The only difference between the reference and hybrid solvers is the replacement of the physical junction model K by its neural approximation K_θ, while the numerical scheme, initial conditions, and external boundary conditions are kept identical. This ensures that any discrepancy between their trajectories at the observed locations can be attributed solely to this substitution.

This training strategy requires repeatedly simulating the full network forward in time and propagating gradients through the numerical solver. Following terminology used in related work, we refer to this approach as *sweeping training*, as the simulator sequentially sweeps through time to generate the full state trajectory. This stands in contrast to the previous training scenarios, where the loss could be evaluated locally at individual time steps without simulating the entire system.

As a consequence, training on sparse observations is substantially more computationally expensive than both direct flux fitting and full-dataset training. However, this approach makes it possible to learn the junction model in settings with incomplete measurements, where pointwise supervision of state updates is not available.

Let $\hat{\mathcal{T}} = \{0, 1, \dots, \hat{M}\}$ denote the discrete time grid of the hybrid solver. Running the hybrid solver produces a complete hybrid state $\{\hat{\rho}_{r,j}^{\hat{k}}\}_{r\in\mathcal{R},\, j\in\mathcal{X}_r,\, \hat{k}\in\hat{\mathcal{T}}}$. Recall that the time step in the finite volume scheme depends on the current approximate solution through the global CFL condition (63). Since the reference and hybrid solvers generally produce different solutions, they will in general use different time step sizes, and their internal time grids will not coincide. In particular, the physical times t_k of the reference solution do not necessarily satisfy $t_k = \hat{t}_k$, not even $t_k = \hat{t}_{\hat{k}}$ for any $\hat{k}$.

To ensure that the hybrid solver reaches the observed times t_k with $k \in \mathcal{T}_{\text{obs}}$, the hybrid time step is chosen as the minimum of the CFL-restricted step size and the remaining time until the next observation. Specifically, at hybrid time level $\hat{k}$ the time increment is given by

$$\Delta t_{\hat{k}} = \min\Big(\Delta t_{\hat{k}}^{\text{CFL}},\ t_{k^+} - \hat{t}_{\hat{k}}\Big),$$

where $k^+ \in \mathcal{T}_{\text{obs}}$ denotes the smallest observed index such that $t_{k^+} > \hat{t}_{\hat{k}}$.

The sparse MSE loss is then defined by comparing the hybrid and reference solutions only at the observed locations and observation times,

$$\mathcal{L}_{\text{sparse}}(\theta) = \frac{1}{|\mathcal{C}_{\text{obs}}|\,|\mathcal{T}_{\text{obs}}|} \sum_{k\in\mathcal{T}_{\text{obs}}} \sum_{(r,j)\in\mathcal{C}_{\text{obs}}} \big\|\rho_{r,j}^{k} - \hat{\rho}_{r,j}^{\hat{k}(k)}\big\|_2^2, \tag{77}$$

where $\hat{k}(k)$ denotes the internal time index of the hybrid solver corresponding to the observation time t_k, that is, $\hat{t}_{\hat{k}(k)} = t_k$.

We remark that one could alternatively enforce a fixed global time step based on a uniform CFL restriction, for example by choosing $\Delta t = L/N \min_{r\in\mathcal{R}} v_{i,\max} \le \Delta t_{CFL}^k$, which guarantees stability of the reference solver for the Rusanov flux in the present configuration. Since the reference solver satisfies the physical constraints by construction, such a choice ensures that the CFL condition is always respected in the reference simulation.

In the hybrid solver, however, the junction fluxes are produced by the neural junction model and are not constrained to remain physically admissible. If the hybrid solver produces densities outside the physical range, the resulting wave speeds increase, which in turn leads to a more restrictive CFL condition and smaller allowable time steps. As a result, enforcing a globally fixed time step does not prevent numerical difficulties once unphysical states occur.

For this reason, we retain the adaptive time-stepping strategy described above, which adjusts the time step dynamically while ensuring that the solver reaches the observation times.

Road	Role	bL [m]	$\rho_{\max}$	$v_{\max}$ [km/h]	$\rho_0(x)$	$\rho_i^{\mathrm{ext}}(t)$
1	in	50	1	60	$0.2 + 0.2\,x$	0.25
2	in	50	1	60	$0.15 + 0.25\,\exp\bigl(-40(x-0.5)^2\bigr)$	$0.15 + 0.05\sin(0.2t)$
3	out	50	1	60	0.05	–
4	out	50	1	60	0.10	–

Table 4: Road-specific parameters for the simple 2-to-2 junction. Inflow fluxes use the same $v_{\max}$ and $\rho_{\max}$ as the road they enter.

We remark that the sparse loss $\mathcal{L}_{\mathrm{sparse}}$ in (77) treats all observed density values equally. Since the traffic densities are normalized, a deviation of the same magnitude contributes equally to the loss, regardless of whether it occurs on a road with different speed limits or number of lanes. In particular, the loss does not account for the fact that errors on some roads may have a larger impact on the overall traffic dynamics than others.

In principle, this could be addressed by introducing road-dependent weighting factors in front of the local quadratic loss terms, for example to reflect differences in speed limits or effective road capacities. We do not pursue such weighting strategies here.

5.3.1 Comparison of the three training scenarios

As a final experiment, we consider a road–network setting in which a neural junction model is trained using each of the three learning strategies introduced above. Specifically, we compare supervised flux fitting, full-state training, and training from sparse observations, and study how the choice of training regime affects the learned junction behavior and the resulting network dynamics.

Training under sparse observations fundamentally changes the learning procedure. As a consequence, backpropagation must pass through the entire time-dependent finite volume solver, which significantly increases both memory usage and runtime. To keep the experiment computationally feasible while still capturing the essential behavior of sparse training, we therefore consider a simplified 2-to-2 junction with short roads and a coarse spatial discretization.

We consider a single 2-to-2 junction with two incoming and two outgoing roads, and assume that traffic flow is governed by the system defined in Section 2.6. The junction is equipped with the physical junction model K defined in (30), using the fixed distribution matrix

$$A = \begin{bmatrix} 0.25 & 0.75 \\ 0.75 & 0.25 \end{bmatrix}.$$

The corresponding speed limits, road lengths, maximal densities, initial density profiles, and external boundary inflow fluxes defining the reference configuration are summarized in Table 3.

To generate reference data, we use the first-order Rusanov finite volume road–network solver described in Section 4.2. Each road is using $N = 4$ finite volume cells. The reference simulation is performed over a time horizon of $T = 10\,\mathrm{s}$ using the physical junction model K.

To train a neural junction model from sparse observations, we define an observation set as described in Section 5.3. The observation locations are chosen as one fixed cell on each road, corresponding to the normalized spatial position $x = 0.4$. Observations are taken at integer times, yielding one measurement per second throughout the simulation. This sparse measurement setup mimics a situation in which only a small number of fixed sensors are available in the network.

For the comparison across training regimes, we use the same neural network architecture for all three junction models. Specifically, the flux model $K_{\theta,\mathrm{flux}}$, the full-dataset model $K_{\theta,\mathrm{full}}$, and the sparse-observation model $K_{\theta,\mathrm{sparse}}$ are all approximated by a fully connected feedforward neural network with four hidden layers, each of width 32. The input and output dimensions are both equal to 5, corresponding to the total number of incoming and outgoing roads at the junction. All hidden layers use ReLU activation functions (66), while the output layer is linear.

Figure 16: Incomplete state training loss for the neural network approximation of the physical junction model K.

Model	Junction MSE	Sparse-location MSE
Flux-trained	2.0×10^{-8}	6.1×10^{-9}
Full-state	5.0×10^{-4}	4.0×10^{-4}
Sparse	8.1×10^{-6}	2.2×10^{-6}

Table 5: Simulation losses evaluated at junction-adjacent cells and at sparse observation locations.

The flux model $K_{\theta,\text{flux}}$ is trained by minimizing the flux-based loss $\mathcal{L}_{\text{flux}}$ (73), while the full-dataset model $K_{\theta,\text{full}}$ is trained by minimizing the density-based loss $\mathcal{L}_{\text{full}}$ (76). In both cases, training is performed using a full-batch Adam optimizer (70) with learning rate $\alpha = 10^{-3}$ for 10 000 epochs.

A practical difficulty in sparse-observation training is that, during the early stages of optimization, a poorly trained junction model may produce unrealistic fluxes. This can lead to overly restrictive CFL conditions (63), significantly increasing the computational cost, through extra time steps. To mitigate this issue, the sparse-observation model $\hat{K}_{\theta,\text{sparse}}$ is initialized using a pretrained baseline junction model. The baseline model is obtained through supervised training on a synthetic dataset, generated from using a uniform turning fraction matrix, with entries $a_{ij} = 1/2$. A dataset of 1 000 admissible junction density states $\boldsymbol{\rho}_K$ and corresponding junction fluxes is constructed, and a neural junction model is trained by minimizing the flux loss $\mathcal{L}_{\text{flux}}$ (73). The training uses 2 000 epochs, and learning rate $\alpha = 10^{-3}$.

Starting from this initialization, the parameters are further optimized by minimizing the sparse loss $\mathcal{L}_{\text{sparse}}(\theta)$ (77). Due to the high computational cost of evaluating this loss, sparse-observation training is performed for only 10 epochs. The corresponding training loss over these epochs is shown in Figure 16. The loss decreases consistently over all epochs, indicating that the training procedure is effective, but 10 epochs are clearly insufficient for convergence of the junction model.

We evaluate the three learned junction models $K_{\theta,\text{flux}}$, $K_{\theta,\text{full}}$, and $K_{\theta,\text{sparse}}$ by embedding them into the same hybrid road–network solver used to generate the reference data. Model performance is assessed using the sparse loss $\mathcal{L}_{\text{sparse}}$ defined in (77), evaluated at one measurement per second. To distinguish between junction-level and network-level accuracy, the loss is computed at the junction-adjacent cells and at the fixed interior observation cells used during sparse-observation training. This allows a direct comparison of the three training regimes under identical evaluation conditions.

Table 5 shows clear and consistent differences between the three training strategies. The flux-trained model performs best overall, with the lowest MSE both at the junction-adjacent cells and at the sparse observation locations. This is expected, since the model is trained directly to reproduce the junction fluxes.

The sparse-observation model also achieves low errors in both settings. Its errors are larger than those of the flux-trained model, but remain several orders of magnitude smaller than those of the

full-dataset model. In contrast, the full-dataset model performs worst, with substantially higher errors at both junction and interior locations.

The relatively good performance of the flux-trained and sparse-observation models away from the junction can be partly explained by the locality of the finite volume scheme. Over a single time step, the update of a cell depends only on its immediate neighbors. As a result, inaccuracies introduced at the junction affect interior cells only gradually through successive updates. Over the short simulation horizon considered here, this limits the propagation of junction errors into the interior.

The poor performance of the full-dataset model may appear counterintuitive, since the training converges and the loss (76) directly penalizes discrepancies between the physical and learned junction fluxes. Indeed, the full-dataset loss is proportional to the MSE between $K(\boldsymbol{\rho}_K^k)$ and $\hat{K}_\theta(\boldsymbol{\rho}_K^k)$ at the junction-adjacent cells, and one would therefore expect accurate identification of the junction model.

The observed behavior suggests that the difficulty lies not in optimization failure, but in the nature of the learning problem itself. Viewed from an inverse-problem perspective, the task is to recover an unknown junction model from indirect and limited observations. Although the loss becomes small, it constrains the junction model only through a scalar-weighted error evaluated at the finite set of junction states visited in the data. This may be sufficient to fit the loss, but not to uniquely or robustly determine the junction model as a function on the relevant state space. As a result, convergence of the training objective does not necessarily imply good performance in forward simulation.

Using a similar procedure to that presented in [5], one can derive an alternative local loss by exploiting the structure of the finite volume update rule (40) at junction-adjacent cells. Consider a junction-adjacent cell (r, j_r^J) on an incoming road. Rearranging the update shows that the junction flux can be expressed explicitly as

$$K(\boldsymbol{\rho}_K^k)_r = F^k_{r,j_r^J-1/2} - \frac{\Delta x_r}{\gamma_r \Delta t_k}\big(\rho^{k+1}_{r,j_r^J} - \rho^k_{r,j_r^J}\big).$$

This provides an explicit target value for the junction model at the observed junction state $\boldsymbol{\rho}_K^k$. Replacing the physical junction model K by its neural approximation $\hat{K}_\theta$ then leads to the local loss

$$\ell^k_{r,j_r^J}(\theta) = \left|\hat{K}_\theta(\boldsymbol{\rho}_K^k)_r - \left[F^k_{r,j_r^J-1/2} - \frac{\Delta x_r}{\gamma_r \Delta t_k}\big(\rho^{k+1}_{r,j_r^J} - \rho^k_{r,j_r^J}\big)\right]\right|^2,$$

which compares the learned junction flux directly to a quantity computed from observed state variables, rather than indirectly through state updates as in (75). Notice that for the sparse training method this loss is not possible, as we are not guaranteed for any density observations at the junction. It is worth noting that, in the sparse-observation setting, both sides of the loss involve simulated quantities rather than a direct comparison between a reconstructed junction flux and an observed target, which makes the operator-based loss unavailable.

Beyond these locality effects, differences in the training objectives also play an important role. The relatively good performance of the sparse-observation model may be explained by the fact that its training objective depends on forward simulations over multiple time steps. Although junction fluxes are not observed directly, the loss reflects how the learned junction model influences the simulated state over time, which may provide a more informative training signal than one-step updates alone.

Finally, the observed performance differences may also be influenced by implementation details. In the present setup, junction fluxes are imposed indirectly via ghost cells (see Section 3.6) rather than applied explicitly at the junction interface. In the full-dataset setting, the neural junction model does not take ghost-cell values as explicit inputs, and the loss is evaluated only on the resulting state updates. This indirect coupling may further weaken the effective training signal and contribute to the reduced performance of the full-dataset model.

6 Conclusion

This report investigated the use of neural networks as junction models in traffic–flow networks governed by hyperbolic conservation laws. The objective was to assess whether data–driven junction couplings can be learned reliably and embedded into finite volume solvers, and to study how the availability and structure of observations influence the learning process.

A hybrid solver framework was introduced in which the analytical junction model is replaced by a neural network, while the LWR equations on the roads and the finite volume discretization are kept unchanged. Three training scenarios were considered: direct supervision using junction fluxes, indirect training based on full state observations, and training with sparse measurements. The numerical experiments demonstrate that direct junction–level supervision yields stable and accurate learning, whereas indirect training through the numerical solver is considerably more sensitive to the loss formulation and observation regime.

One key finding is that increased data availability does not necessarily lead to improved performance when the training signal becomes indirect. In particular, the full–dataset training scenario performed significantly worse than expected, despite convergence of the training objective. This behavior suggests that the difficulty lies in the identifiability of the junction model rather than in optimization failure. By contrast, the sparse–observation setting was shown to admit accurate learning, provided that the training objective captures the forward influence of the junction coupling on the network dynamics.

Several directions for future work follow from these observations. A natural next step is to better understand the behavior observed in the different training scenarios, in particular the strong performance of the sparse–observation approach compared to the full–dataset setting. Although the full state of the system is available in the latter case, the resulting training signal appears to be less effective than the indirect supervision provided by sparse observations over multiple time steps. Further analytical and numerical investigation is needed to clarify the mechanisms behind this behavior.

In particular, it would be of interest to study how properties of the numerical scheme and the training objective influence the sensitivity of the learned junction model to errors and perturbations. This includes examining the role of the finite volume update structure, the choice of loss function, the time horizon over which the training signal is accumulated, and implementation choices such as the indirect imposition of junction fluxes via ghost cells. Such analysis may help explain why certain training strategies lead to more reliable forward simulations than others.

From a computational perspective, the efficiency of the training procedure could be improved through more optimized implementations of the hybrid solver. Exploring alternative optimization strategies, neural network architectures, or training procedures may further improve robustness, particularly in the sparse–observation regime. Additional extensions include relaxing the assumption of known initial and boundary conditions and considering more complex junction configurations or larger networks.

Overall, the results highlight both the potential and the limitations of learning junction models from data. They emphasize that successful integration of machine learning into conservation–law solvers depends not only on data availability, but also on how the learning problem interacts with the numerical structure of the underlying solver.

References

[1] Mauro Garavello and Benedetto Piccoli. *Traffic Flow on Networks: Conservation Law Models*. Macroscopic models for traffic on networks, including junction coupling rules and conservation law frameworks. American Institute of Mathematical Sciences, 2006. ISBN: 1-60133-000-0.

[2] Giuseppe Maria Coclite, Mauro Garavello and Benedetto Piccoli. 'Traffic flow on a road network'. In: *SIAM Journal on Mathematical Analysis* 36.6 (2005). Fluid-dynamic conservation law model on road networks; junctions underdetermined without fixed distribution rules., pp. 1862–1886. DOI: 10.1137/S0036141004402683.

[3] M. Raissi, P. Perdikaris and G. E. Karniadakis. 'Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations'. In: *Journal of Computational Physics* 378 (2019), pp. 686–707.

[4] Jiawei Lu et al. 'Physics-informed neural networks for integrated traffic state and queue profile estimation: A differentiable programming approach on layered computational graphs'. In: *Transportation Research Part C* (2023). DOI: 10.1016/j.trc.2023.104224.

[5] Bendik Skundberg Waade. 'Numerics-Informed Neural Networks and Inverse Problems with Hyperbolic Balance Laws'. Master's thesis. Trondheim, Norway: Norwegian University of Science and Technology (NTNU), June 2023.

[6] Martin Treiber and Arne Kesting. *Traffic Flow Dynamics: Data, Models and Simulation*. Springer, 2013.

[7] Dirk Helbing. 'Traffic and related self-driven many-particle systems'. In: *Reviews of Modern Physics* 73.4 (2001), pp. 1067–1141.

[8] M. J. Lighthill and G. B. Whitham. 'On kinematic waves. II. A theory of traffic flow on long crowded roads'. In: *Proceedings of the Royal Society of London. Series A* 229.1178 (1955), pp. 317–345.

[9] P. I. Richards. 'Shock waves on the highway'. In: *Operations Research* 4.1 (1956), pp. 42–51.

[10] Randall J. LeVeque. *Finite Volume Methods for Hyperbolic Problems*. Cambridge Texts in Applied Mathematics. Cambridge University Press, 2002.

[11] T. Helset. 'Speed Limit and Traffic-Light Control for Traffic-Flow Networks'. Master's thesis. Trondheim, Norway: Norwegian University of Science and Technology (NTNU), 2024.

[12] A. Aw and M. Rascle. 'Resurrection of second order models of traffic flow'. In: *SIAM Journal on Applied Mathematics* 60.3 (2000), pp. 916–938.

[13] H. M. Zhang. 'A non-equilibrium traffic model devoid of gas-like behavior'. In: *Transportation Research Part B* 36.3 (2002), pp. 275–290.

[14] H. Greenberg. 'An analysis of traffic flow'. In: *Operations Research* 7.1 (1959), pp. 79–85.

[15] R. T. Underwood. 'Speed, volume, and density relationships: Quality and theory of traffic flow'. In: *Yale Bureau of Highway Traffic* (1961), pp. 141–188.

[16] G. F. Newell. 'A simplified theory of kinematic waves in highway traffic, part I: General theory'. In: *Transportation Research Part B* 27.4 (1993), pp. 281–287.

[17] Siddhartha Mishra, Ulrik Skre Fjordholm and Rémi Abgrall. *Numerical Methods for Conservation Laws and Related Equations*. Lecture notes / course material, 2016.

[18] S. K. Godunov. 'A Difference Scheme for Numerical Solution of Discontinuous Solutions of Hydrodynamic Equations'. In: *Mathematicheskii Sbornik* 47 (1959), pp. 271–306.

[19] Peter D. Lax and Kurt O. Friedrichs. 'Systems of Conservation Equations with a Convex Extension'. In: *Proceedings of the National Academy of Sciences of the United States of America* 40.5 (1954), pp. 327–331. DOI: 10.1073/pnas.40.5.327.

[20] V. S. Rusanov. 'Calculation of Shocks and Rarefaction Waves for Hyperbolic Systems'. In: *Zhurnal Vychislitel'noi Matematiki i Matematicheskoi Fiziki* 1.3 (1961).

[21] Kurt Hornik. 'Approximation capabilities of multilayer feedforward networks'. In: *Neural Networks* 4.2 (1991), pp. 251–257.

[22] Michael Herty and Niklas Kolbe. 'Data-driven models for traffic flow at junctions'. In: *Mathematical Methods in the Applied Sciences* 47.11 (2024), pp. 8946–8968. DOI: 10.1002/mma.10053.

[23] Diederik P. Kingma and Jimmy Ba. 'Adam: A Method for Stochastic Optimization'. In: *Proceedings of the International Conference on Learning Representations (ICLR)*. 2015.

[24] Andreas Griewank and Andrea Walther. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*. 2nd ed. SIAM, 2008.

[25] Adam Paszke et al. 'PyTorch: An Imperative Style, High-Performance Deep Learning Library'. In: *Advances in Neural Information Processing Systems* 32 (2019).

[26] Pratik Rathore et al. 'Challenges in Training Physics-Informed Neural Networks: A Loss Landscape Perspective'. In: *arXiv preprint arXiv:2402.01868* (2024). ICML 2024 Oral. DOI: 10.48550/arXiv.2402.01868.

[27] I. T. Jolliffe and J. Cadima. *Principal Component Analysis*. 2nd ed. Springer Series in Statistics. Springer, 2016.